



# **UnionPay Integrated Circuit Card Specifications**

## **— Basic Specifications**

### **Part IV Debit/Credit Application Security Specification**

**Version 2017**

**THIS PAGE IS INTENTIONALLY LEFT BLANK.**

## Table of Contents

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>SUMMARY OF REVISIONS.....</b>                                 | <b>1</b>  |
| <b>1 APPLICATION SCOPE .....</b>                                 | <b>2</b>  |
| <b>2 REFERENCES .....</b>                                        | <b>3</b>  |
| <b>3 OFF-LINE DATA AUTHENTICATION.....</b>                       | <b>4</b>  |
| 3.1 KEY AND CERTIFICATE .....                                    | 5         |
| 3.1.1 Certification Authority .....                              | 6         |
| 3.1.2 Public and Private key pair.....                           | 6         |
| 3.2 STATIC DATA AUTHENTICATION (SDA) .....                       | 7         |
| 3.2.1 Key and Certificate .....                                  | 9         |
| 3.2.2 Obtain Certification Authority public key .....            | 12        |
| 3.2.3 Obtain Issuer Public Key .....                             | 12        |
| 3.2.4 Authentication of Signed Static Application Data .....     | 14        |
| 3.3 DYNAMIC DATA AUTHENTICATION(DDA).....                        | 15        |
| 3.3.1 Key and Certificate .....                                  | 18        |
| 3.3.2 Obtain Certification Authority Public Key.....             | 22        |
| 3.3.3 Obtain Issuer Public Key .....                             | 22        |
| 3.3.4 Obtain IC Card Public Key.....                             | 24        |
| 3.3.5 Standard DDA.....                                          | 26        |
| 3.3.6 Combined DDA/Application Cryptogram Generation (CDA) ..... | 29        |
| <b>4 APPLICATION CRYPTOGRAM AND ISSUER AUTHENTICATION.....</b>   | <b>41</b> |
| 4.1 APPLICATION CRYPTOGRAM GENERATION .....                      | 41        |
| 4.1.1 Selection of data source .....                             | 41        |
| 4.1.2 Application Cryptogram Algorithm.....                      | 42        |
| 4.2 ISSUER AUTHENTICATION.....                                   | 42        |
| 4.3 KEY MANAGEMENT.....                                          | 43        |
| <b>5 SECURITY MESSAGE.....</b>                                   | <b>44</b> |
| 5.1 MESSAGE FORMAT .....                                         | 44        |
| 5.2 SECURE MESSAGE INTEGRITY AND AUTHENTICATION .....            | 44        |
| 5.2.1 Command Data Field.....                                    | 44        |
| 5.2.2 MAC process Key Generation.....                            | 44        |
| 5.2.3 MAC Calculation .....                                      | 44        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 5.3      | MESSAGE CONFIDENTIALITY.....                                 | 45        |
| 5.3.1    | Command Data Field.....                                      | 45        |
| 5.3.2    | Encrypting Process Key Generation .....                      | 45        |
| 5.3.3    | Encryption and Decryption .....                              | 45        |
| 5.4      | KEY MANAGEMENT.....                                          | 45        |
| <b>6</b> | <b>CARD SECURITY.....</b>                                    | <b>46</b> |
| 6.1      | SYMBIOTIC APPLICATION .....                                  | 46        |
| 6.2      | KEY INDEPENDENCE.....                                        | 46        |
| 6.3      | INTERNAL SECURITY SYSTEM OF CARD.....                        | 46        |
| 6.3.1    | Internal Security Target of Card.....                        | 46        |
| 6.3.2    | Internal Security Overview of Card .....                     | 46        |
| 6.3.3    | File Control Information.....                                | 47        |
| 6.3.4    | File Control Parameters .....                                | 50        |
| 6.3.5    | Recommended Access Conditions for Local Data on IC Card..... | 51        |
| 6.4      | KEY TYPES IN CARD .....                                      | 51        |
| <b>7</b> | <b>TERMINAL SECURITY.....</b>                                | <b>53</b> |
| 7.1      | SECURITY REQUIREMENTS FOR TERMINAL DATA.....                 | 53        |
| 7.1.1    | General Requirements.....                                    | 53        |
| 7.1.2    | Physical Security Requirement of Security.....               | 54        |
| 7.1.3    | Logical Security Requirement of Security Module .....        | 54        |
| 7.2      | SECURITY REQUIREMENTS ON TERMINAL DEVICES.....               | 54        |
| 7.2.1    | Anti-intrusion Devices.....                                  | 54        |
| 7.2.2    | PINPAD Security.....                                         | 56        |
| 7.3      | REQUIREMENTS ON TERMINAL KEY MANAGEMENT .....                | 57        |
| 7.3.1    | Terminal Key Type.....                                       | 57        |
| 7.3.2    | Certification Authority Public Key Management .....          | 58        |
| <b>8</b> | <b>KEY MANAGEMENT SYSTEM.....</b>                            | <b>61</b> |
| 8.1      | CERTIFICATION AUTHORITY PUBLIC KEY MANAGEMENT.....           | 61        |
| 8.1.1    | Life Cycle of Certification Authority Public Key .....       | 61        |
| 8.1.2    | Compromise of Certification Authority Public Key.....        | 63        |
| 8.1.3    | Strategies of Certification Authority Key Management.....    | 64        |
| 8.2      | MANAGEMENT OF ISSUER PUBLIC KEY .....                        | 67        |
| 8.3      | MANAGEMENT OF ISSUER SYMMETRICAL KEY .....                   | 68        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 8.3.1 Requirement of Security .....                           | 68        |
| 8.3.2 Functional Requirements .....                           | 68        |
| <b>9 SECURITY SYSTEMS.....</b>                                | <b>70</b> |
| 9.1 SYMMETRIC ENCRYPTION SYSTEM.....                          | 70        |
| 9.1.1 Encryption and Decryption.....                          | 70        |
| 9.1.2 Message Authentication Code.....                        | 71        |
| 9.1.3 Session Key Generation .....                            | 73        |
| 9.1.4 Derivation of Sub-Key.....                              | 75        |
| 9.2 ASYMMETRICAL ENCRYPTION SYSTEM .....                      | 76        |
| 9.2.1 Digital Signature Scheme for Message Recovery.....      | 76        |
| <b>10 APPROVED ALGORITHM .....</b>                            | <b>78</b> |
| 10.1 SYMMETRIC ENCRYPTION ALGORITHM .....                     | 78        |
| 10.1.1 DES .....                                              | 78        |
| 10.1.2 SSF33 .....                                            | 78        |
| 10.2 ASYMMETRICAL ENCRYPTION ALGORITHM.....                   | 78        |
| 10.2.1 RSA.....                                               | 78        |
| 10.3 HASH ALGORITHM .....                                     | 80        |
| 10.3.1 SHA-1 .....                                            | 80        |
| <b>APPENDIX A (INFORMATIVE APPENDIX)MAC CALCULATION .....</b> | <b>81</b> |
| A.1 CALCULATION OF MAC/TA .....                               | 81        |

## Summary of Revisions

The changes listed below are associated with the current version.

| Change Description                                                                                                                                                                                     | Revised/<br>Deleted/<br>Added | Where to look    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|------------------|
| Added Signed Data Format '95' in in Table 13 and Table 15, and supplemented related requirements of the generation and verification of dynamic data signature to support ODA for online authorization. | Added                         | 3.3.5.1, 3.3.5.2 |
| Revised the description of derivation of sub-key to keep consistency with Part II                                                                                                                      | Revised                       | 9.1.4            |

## **1 Application Scope**

This book applies to all UPI Participants.

## 2 References

The terms and conditions of the following documents have become the terms and conditions of this book. The modification list (excluding corrected contents) or revised edition attached to the dated documents shall not apply to this book. However, Participants may study whether to apply the latest versions of such documents. The latest versions of non-dated documents shall apply to this book.

EMV Integrated Circuit Card Specifications for Payment Systems, Version 4.3 Book 2 Security\_and\_Key\_Management.

ISO/IEC 7816-4:2005      *Identification cards -- Integrated circuit cards -- Part 4: Organization, security and commands for interchange*

ISO/IEC 7816-5:1994/Amd 1:1996 *Identification cards -- Integrated circuit(s) cards with contacts -- Part 5: Numbering system and registration procedure for application identifiers*

ISO 13491-2:2005      *Banking -- Secure cryptographic devices (retail) -- Part 2: Security compliance checklists for devices used in financial transactions*

ISO 8731-1              *Banking -- Approved algorithms for message authentication -- Part 1: DEA*

ISO 8732                *Banking -- Key management (wholesale)*

ISO/IEC 9796-2      *Information technology -- Security techniques -- Digital signature schemes giving message recovery -- Part 2: Integer factorization based mechanisms*

ISO/IEC 9797-1      *Information technology -- Security techniques -- Message Authentication Codes (MACs) -- Part 1: Mechanisms using a block cipher*

ISO/IEC 10116      *Information technology -- Security techniques -- Modes of operation for an n-bit block cipher*

ISO 13491-1            *Banking -- Secure cryptographic devices (retail) -- Part 1: Concepts, requirements and evaluation methods*



### 3 Offline Data Authentication

Offline data authentication is a method used by the terminal to authenticate card data based on public key technique. There are two types of offline data authentication:

- Static data authentication (SDA)
- Dynamic data authentication (DDA)

In the process of SDA, the terminal verifies the legality of the static data on the card where SDA is capable of confirming that issuer application data has not been illegally tampered since the card personalization.

In the process of DDA, the terminal verifies the static data on the card and signature for relevant transaction information generated from the card. DDA is capable of confirming that the issuer application data on the card has not been illegally tampered since the card personalization. DDA can also verify the authenticity of the card and prevent it from being illegally reproduced. DDA can be standard dynamic data authentication or combined dynamic data authentication/application cryptogram generation (CDA). AIP points out the offline data authentication method supported by the IC card.

Result of the offline data authentication will determine whether the card and terminal will execute offline transaction, online authorization, or decline transaction offline. Comparison of SDA and DDA is listed in Table 1.

Table 1 Comparison of SDA and DDA

|                                                                    | <b>SDA</b> | <b>DDA</b> |
|--------------------------------------------------------------------|------------|------------|
| Confirm that the card data have not been tampered                  | Yes        | Yes        |
| Prevent the card from being copied                                 | No         | Yes        |
| Requires the card to support asymmetrical encryption algorithm     | No         | Yes        |
| Requires the terminal to support asymmetrical encryption algorithm | Yes        | Yes        |
| Contain issuer public key certificate                              | Yes        | Yes        |
| Contain card public key certificate                                | No         | Yes        |
| Times of public key decryption                                     | 2          | 3          |

Offline data authentication performs only one authentication method with the priorities of the three offline authentication methods (from highest to lowest) as: CDA, standard DDA and SDA. Circumstances and the offline verification modes supported by card and terminal are listed in Table 2.

Table 2: Priorities of SDA, DDA, and CDA Processing

| AIP indicates card support | Terminal supports SDA | Terminal supported SDA and standard DDA | Terminal supported SDA, standard DDA and CDA |
|----------------------------|-----------------------|-----------------------------------------|----------------------------------------------|
| SDA                        | SDA                   | SDA                                     | SDA                                          |
| SDA and standard DDA       | SDA                   | Standard DDA                            | Standard DDA                                 |
| SDA, standard DDA and CDA  | SDA                   | Standard DDA                            | CDA                                          |

Records used in offline data authentication must be in TLV encoding format and Tag = '70'. Data in records used in offline data authentication depend on the SFI of the files:

- For files with SFI of 1-10, the Tag ('70') of the record and the length of record are not used in offline data authentication processing, all other data in command READ RECORD response data field (except SW1 and SW2) participate in the offline data authentication.
- For files with SFI of 11 to 30, the Tag ('70') of the record and the length of record are used in offline data authentication processing; therefore, all data in command READ RECORD response data field (except SW1 and SW2) participate in offline data authentication.
- If the Tag of the record in the files used in offline data authentication is not '70', it means that the offline data authentication has performed and failed. The terminal must configure "offline data authentication executed digit" of TSI, "offline static data authentication failed digit" corresponding to TVR, and "offline dynamic data authentication failed digit" or "CDA failed" digit.

### 3.1 Key and Certificate

The terminal realizes offline data authentication by verifying the signature and certificate on the IC card with public key algorithm. Public key technique uses private key to generate encrypted data (certificate or signature) which can be decrypted with the public key for verification and data recovery. The bit length of RSA public key modulus should be a multiple of 8 and the leftmost (highest) bit of the leftmost (highest) byte should be 1. All lengths should be in byte.

If the static application data on the card are not unique (for example, the card adopts different CVMs for international and domestic transactions), the card must support multi IC card public key certificates (or static data signatures). If the signed static application data may be modified after the card is issued, the card must support the update of IC card public key certificate (or static data signature).

### **3.1.1 Certification Authority**

Offline data authentication requires a certification authority (CA) and the certification authority should have encryption devices with high security grade to sign issuer public key certificates. Every terminal consistent with *UnionPay Integrated circuit card specifications-Basic Specifications* should save corresponding certification authority public keys for every application it recognizes.

### **3.1.2 Public and Private key pair**

Certification authority and the Issuer must use the asymmetrical algorithm designated in Section 10.2 to generate certification authority public and private key pair, issuer public and private key pair, and IC card public and private key pair. Description of offline data authentication process and related data elements in this chapter is based on RSA algorithm.

#### **3.1.2.1 Certification Authority Public and Private Key Pair**

Certification authority will generate a maximum of 6 public and private key pairs and every public and private key pair will be allocated with a unique index of certification authority public key. The certification authority public key and the index are loaded to terminal by Acquirer, while the certification authority private key is kept by the certification authority to ensure its confidentiality and security.

The terminal must have sufficient space to store certification authority public keys and the corresponding registered application provider identifiers (RID) and certification authority public key index. The terminal locates certification authority public key via RID and certification authority public key index.

Length of certification authority public keys must be within the scope defined in Section 10.2.1 and the certification authority public key index must be equal to 3 or  $2^{16}+1$ .

#### **3.1.2.2 Issuer Public and Private Key Pair**

To Support SDA or DDA requires the Issuer to generate issuer public and private key pair and obtain issuer public key certificate from certification authority. The Issuer transmits the public key to the certification authority, and the certification authority uses the certification authority private key with length of modulus longer than or equal to that of the issuer public key modulus and the public key effective date, later than issuer public key effective date, to sign the public key.

IC card must contain issuer public key certificate and use this certificate to verify the certification authority public key index of the issuer certificate. The issuer private key is kept by the Issuer to ensure its confidentiality and security.

Length of issuer public key modulus must be less than or equal to the maximum length of certification authority public key modulus. The length of issuer public key modulus must be within the scope defined in Section 10.2.1. The issuer public key exponent must be equal to 3 or  $2^{16}+1$ .

The terminal locates certification authority public key via registered application provider identifier (RID) and certification authority public key index, which also uses the certification authority public key to recover issuer public key from issuer certificate, then uses issuer public key to recover and verify the issuer application data on the card.

### 3.1.2.3 IC Card Public and Private Key Pair

To support DDA also requires Issuer to generate IC card public and private key pair for every IC card. IC card private key is stored in the IC card secure storage area. IC card public key is signed by issuer private key to generate IC card public key certificate which is stored in card.

The length of IC card public key modulus must be less than or equal to that of issuer public key modulus and within the scope defined in Section 10.2.1. The IC card public key index must equal to 3 or  $2^{16}+1$ .

Terminal locates certification authority public key via registered application identifier (RID) and certification authority public key index, and uses the certification authority public key to resume issuer public key from certification authority public key certificate, then uses issuer public key to resume IC card public key from IC card public key certificate to verify the dynamic signature data of the card.

IC card public key can also be used for offline cryptogram PIN verification, there is no requirement for offline cryptogram PIN in *UnionPay Integrated circuit card specifications-Basic Specification*.

## 3.2 Static Data Authentication (SDA)

The purpose of SDA is to verify the legality of critical static data identified by application file locator (AFL) and listed in optional static data authentication tag list stored in IC card so as to ensure that the issuer data in IC card have not been illegally tampered with since personalization.

After personalization, the IC card supporting SDA should include the following data elements:

- Certification authority public key index: This single byte data element contains a binary number, indicating which of the corresponding certification authority

public key stored in terminal should be used by the terminal to authenticate the IC card.

- Issuer public key certificate: Variable length data element provided by the certification authority to the Issuer. When the terminal verifies the data element, the issuer public key and other data should be authenticated according to the process described in Section 3.2.3.
- Signed static application data: Variable length data element generated by the issuer private key corresponding to the issuer public key authenticated by issuer public key certificate. It is a digital signature of critical static data element stored in IC card, which is specifically described in Section 3.2.4.
- Issuer Public Key Remainder: A variable length data element, the presence of this data element in the IC card is conditional. Further interpretation is provided in Section 3.2.1.
- Issuer Public Key Exponent: A variable length data element provided by Issuer and further interpretation is provided in Section 3.2.1.

In order to support SDA, every terminal should be able to store six certification authority public keys for every registered application provider identifier (RID), and must make the key information related to the key be interrelated to every key (so that the terminal will be able to support multi algorithms in future and it will be allowable to transit from one algorithm to another, please refer to section 7.3). In case that RID and certification authority public key index provided by IC card is given, the terminal should be able to locate such public keys and related information.

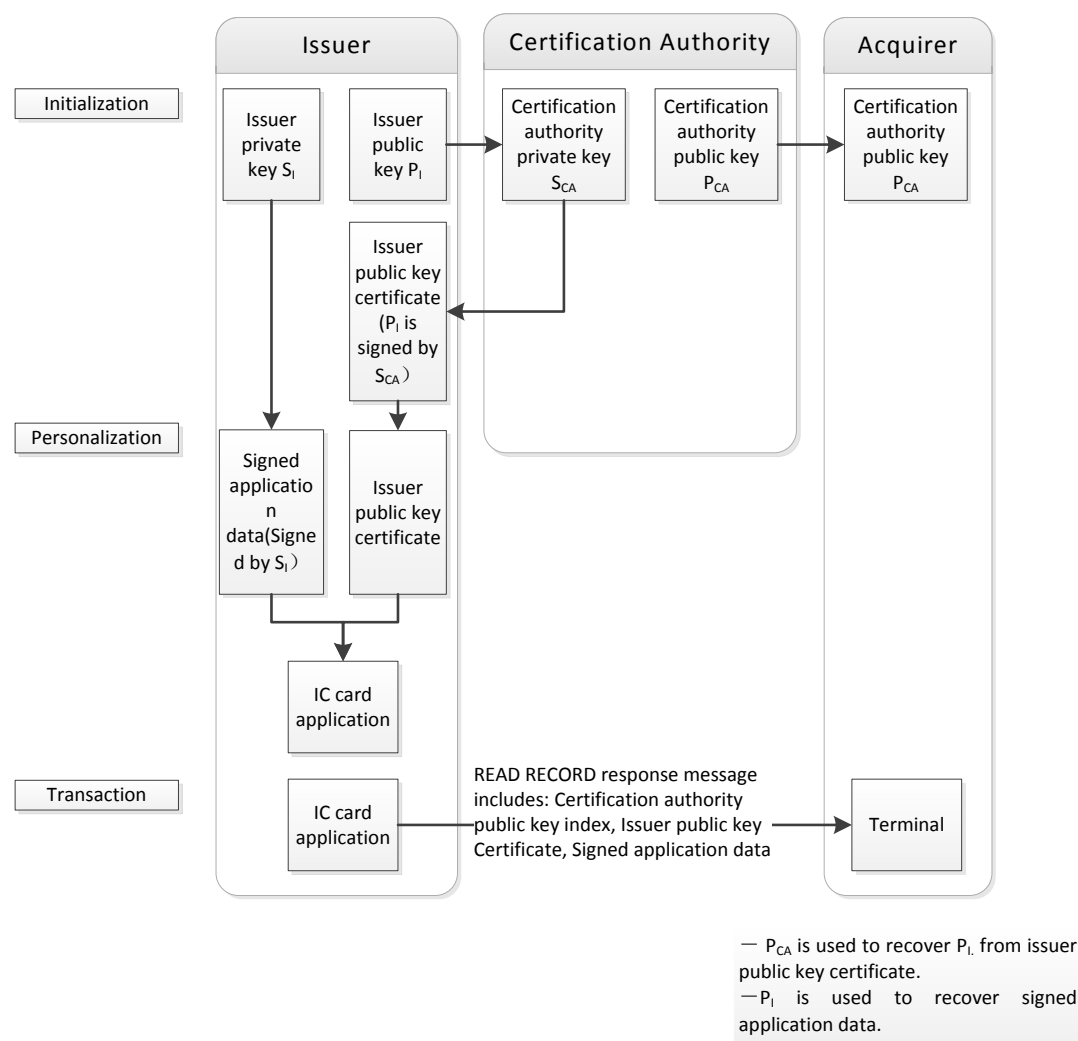


Figure 1 SDA certificate and public key system structure

SDA must use a reversible algorithm indicated in Section 9.2.1 and section 10.2. Section 3.2.1 contains overviews of keys and certificates involved in the process of SDA. The three major steps of authentication process are described in detail in section 3.2.2 and Section 3.2.4:

- The terminal recover certification authority public key.
- The terminal recover issuer public key.
- The terminal verifies signed static application data.

### 3.2.1 Key and Certificate

In order to support SDA, the IC card must contain signed static application data, which is signed via issuer private key. Issuer public key must be stored in the IC card as public key certificate.

To obtain an issue public key certificate, certification authority private key  $S_{CA}$  should be used. The signature scheme in Section 9.2.1 should be used for the data as shown in Table 3.

There is a public key modulus of the public key of certification authority and the public key modulus has  $N_{CA}$  bytes. Public key Index of certification authority must be equal to 3 or  $2^{16}+1$ .

In order to obtain signed static application data, issuer private key  $S_I$  should be used. The signature scheme in Section 9.2.1 should be used for the data shown in Table 4.

There is an issuer public key modulus of the issuer public key pair and the public key modulus has  $N_I$  bytes ( $N_I \leq N_{CA}$ ). If  $N_I > (N_{CA}-36)$ , the issuer public key modulus is divided into two parts, one part contains the high order  $N_{CA}-36$  bytes in public key modulus (the leftmost numbers in issuer public key and the other part contains the surplus low order  $N_I - (N_{CA}-36)$  bytes (the issuer public key remainder). The issuer public key exponent must be equal to 3 or  $2^{16}+1$ .

All information and data required for SDA are explained in Table 5 in detail, which is also stored in the IC card. RID can be obtained from AID while as other information and data can be obtained via reading command record (READ RECORD). SDA will fail if any one of those data is missing.

Table 3 Issuer Public Key Data Signed by Certification Authority (hash Algorithm Input)

| Field Name                             | Length | Description                                                                                               | Format |
|----------------------------------------|--------|-----------------------------------------------------------------------------------------------------------|--------|
| Certificate format                     | 1      | Hexadecimal and the value is '02'                                                                         | b      |
| Issuer identifier                      | 4      | The leftmost 3-8 numbers of the primary account number (hexadecimal number 'F' should be added on right). | cn8    |
| Certificate Expiration Date            | 2      | MMYY, the certificate will be invalid after that date.                                                    | n4     |
| Certificate Serial Number              | 3      | The unique binary number distributed by the certification authority to the certificate                    | b      |
| Hash algorithm identifier              | 1      | The identifier is used for the hash algorithm generating hash result in digital signature scheme.         | b      |
| Issuer public key algorithm identifier | 1      | The identifier used in digital signature algorithm on issuer public key                                   | b      |
| Issuer Public Key                      | 1      | It is used to identify issuer public key                                                                  | b      |

| Length                                                        |                          | length in bytes                                                                                                                                                                                                                                                                                      |   |
|---------------------------------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| Issuer Public Key Exponent Length                             | 1                        | It is used to identify issuer public key exponent length in bytes                                                                                                                                                                                                                                    | b |
| Issuer public key or the leftmost digits of issuer public key | $N_{CA} - 36$            | <p>If <math>N_I \leq N_{CA} - 36</math>, the field contains the whole issuer public key with <math>N_{CA} - 36 - N_I</math> bytes of value 'BB' added on right.</p> <p>If <math>N_I &gt; N_{CA} - 36</math>, the field contains the highest <math>N_{CA} - 36</math> bytes of issuer public key.</p> | b |
| Issuer Public Key Remainder                                   | 0 or $N_I - N_{CA} + 36$ | The field only appears If $N_I > N_{CA} - 36$ and the field contains the lowest $N_I - N_{CA} + 36$ bytes of issuer public key.                                                                                                                                                                      | b |
| Issuer Public Key Exponent                                    | 1 or 3                   | The issuer public key exponent is equal to 3 or $2^{16} + 1$ .                                                                                                                                                                                                                                       | b |

Table 4 Static Application Data Signed by Issuer (i.e. Hash Algorithm Input)

| Field Name                      | Length     | Description                                                                                                                                                                                                                          | Format |
|---------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Signed data format              | 1          | Hexadecimal and the value is '03'                                                                                                                                                                                                    | b      |
| Hash algorithm identifier       | 1          | The identifier is used for the Hash algorithm generating hash result in digital signature scheme.                                                                                                                                    | b      |
| Data authentication code        | 2          | Code distributed by Issuer                                                                                                                                                                                                           | b      |
| Pad pattern                     | $N_I - 26$ | Padding bytes consist of $N_I - 26$ bytes value 'BB'                                                                                                                                                                                 | b      |
| Static data to be authenticated | Variable   | Static data requiring authentication per Section 7.3.1 of <i>UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification</i> . (Please refer to following paragraphs.) | -      |

The input of authentication process consists of records identified by AFL followed by AIP (if AIP is identified by optional static data authentication tag list (tag



‘9F4A’)). If there is a static data authentication tag list, it must only contain tag ‘82’ used for identifying AIP.

Table 5 Data Objects Used in Static Data Authentication

| Tag    | Length              | Value                                                                                                                                                                                      | Format |
|--------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| -      | 5                   | Identifier of registered application provider                                                                                                                                              | b      |
| ‘8F’   | 1                   | Public key index of certification authority                                                                                                                                                | b      |
| ‘90’   | $N_{CA}$            | Issuer public key certificate                                                                                                                                                              | b      |
| ‘92’   | $N_I - N_{CA} + 36$ | Issuer Public Key Remainder(if any)                                                                                                                                                        | b      |
| ‘9F32’ | 1 or 3              | Issuer Public Key Exponent                                                                                                                                                                 | b      |
| ‘93’   | $N_I$               | Signed static application data                                                                                                                                                             | b      |
| -      | Variable            | Static data requiring authentication per Section 7.3.1 of <i>UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification</i> | -      |

### 3.2.2 Obtain Certification Authority public key

The terminal reads the certification authority public key index. In order to use the index and RID, the terminal must confirm and obtain the modulus, index and the related key information of the certification authority public key and the corresponding algorithm to be used. If the terminal hasn’t stored the key related to the index and RID, static data authentication will fail.

### 3.2.3 Obtain Issuer Public Key

1. If the length of issuer public key certificate is different from that of certification authority public key modulus obtained in the previous process, the SDA has failed.
2. In order to obtain the recovered data shown in Table 6, certification authority public key and the corresponding algorithm is used to recover the issuer public key certificate according to the recovery function specified in Section 9.2.1. If the end of recovered data is not equal to ‘BC’, the SDA has failed.

Table 6 Format of Data Recovered from Issuer Public Key Certificate

| Field Name                                                   | Length      | Description                                                                                                                                                                                                                         | Format |
|--------------------------------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Head of recovered data                                       | 1           | Hexadecimal and the value is '6A'                                                                                                                                                                                                   | b      |
| Certificate format                                           | 1           | Hexadecimal and the value is '02'                                                                                                                                                                                                   | b      |
| Issuer identifier                                            | 4           | Leftmost 3-8 numbers of primary account number (adding Hexadecimal number 'F' on the right).                                                                                                                                        | cn8    |
| Certificate Expiration Date                                  | 2           | MMYY, the certificate is invalid after that date.                                                                                                                                                                                   | n4     |
| Certificate Serial Number                                    | 3           | The only binary number distributed by the certification authority to that certificate                                                                                                                                               | b      |
| Identifier of Hash algorithm                                 | 1           | The identifier is used for the Hash algorithm generating hash result in digital signature scheme.                                                                                                                                   | b      |
| Identifier of issuer public key algorithm                    | 1           | It is used in the digital signature algorithm on issuer public key.                                                                                                                                                                 | b      |
| Issuer Public Key Length                                     | 1           | It is used to identify issuer public key length in bytes.                                                                                                                                                                           | b      |
| Issuer Public Key Exponent Length                            | 1           | It is used to identify the byte issuer public key exponent length in bytes.                                                                                                                                                         | b      |
| Issuer public key or the leftmost bytes of issuer public key | $N_{CA}-36$ | If $N_I \leq N_{CA}-36$ , the field contains the whole issuer public key with $N_{CA}-36-N_I$ bytes with value 'BB' added on right. If $N_I > N_{CA}-36$ , the field contains the top order $N_{CA}-36$ bytes of issuer public key. | b      |
| Hash result                                                  | 20          | Issuer public key and Hash value of relevant information                                                                                                                                                                            | b      |
| End of recovered data                                        | 1           | Hexadecimal and the value is 'BC'                                                                                                                                                                                                   | b      |

3. Check the recovered data head. If it is not '6A', the SDA has failed.

4. Check the certificate format. If it is not '02', the SDA has failed.

5. Concatenate from left to right the second to the tenth data elements in Table 6 (from certificate format to the issuer public key or the leftmost byte of issuer public key) and then add the remainder of issuer public key to the end (if any), and final is add the issuer public key exponent.
6. Use the designated hash algorithm (derived from Hash Algorithm Indicator) to calculate the concatenation of the previous step to produce the hash result.
7. Compare the calculated hash result from the previous step with the recovered hash result, if they are different from each other, SDA has failed.
8. Check if the issuer identifier matches the leftmost 3-8 digitals of the primary account number (allowing for the possible padding of the Issuer Identifier with hexadecimal 'F's). If not, the SDA has failed.
9. Verify that the last day of the month specified in the Certificate Expiration Date is equal to or later than today's date. If the Certificate Expiration Date is earlier than today's date, the certificate has expired, in which case SDA has failed.
10. Verify that the concatenation of RID, Certification Authority Public Key Index, and Certificate Serial Number is valid. If not, SDA has failed.
11. If the identifier of issuer public key algorithm cannot be identified, the SDA has failed.
12. If all the checks above are correct, concatenate the Leftmost Digits of the Issuer Public Key and the Issuer Public Key Remainder (if present) to obtain the Issuer Public Key Modulus, and continue with the next steps for the verification of the Signed Static Application Data.

### 3.2.4 Authentication of Signed Static Application Data

1. If the length of signed static application data is different from the length of issuer public key modulus, the SDA has failed.
2. In order to obtain the recovered data specified in Table 7, it is necessary to use issuer public key and corresponding algorithm and apply the recovery function in Section 9.2.1 to the signed static application data. If the end of the recovered data is not equal to 'BC', the SDA has failed.

Table 7 Format of Data Restored from Signed Static Application Data

| Field Name             | Length | Description                       | Format |
|------------------------|--------|-----------------------------------|--------|
| Head of recovered data | 1      | Hexadecimal and the value is '6A' | b      |
| Signed data format     | 1      | Hexadecimal and the value is '03' | b      |

|                              |          |                                                                                                   |   |
|------------------------------|----------|---------------------------------------------------------------------------------------------------|---|
| Identifier of Hash algorithm | 1        | The identifier is used for the hash algorithm generating hash result in digital signature scheme. | b |
| Data verification code       | 2        | Code distributed by Issuer                                                                        | b |
| Pad pattern                  | $N_T-26$ | The pad pattern consists of $N_T-26$ bytes with value 'BB'.                                       | b |
| Hash result                  | 20       | Hash value of static application data requiring authentication                                    | b |
| End of recovered data        | 1        | Hexadecimal and the value is 'BC'                                                                 | b |

3. Check the recovered data head. If it is not '6A', the SDA has failed.
4. Check signed data format. If it is not '03', the SDA has failed.
5. Concatenate from left to right the second to the fifth data elements in Table 7 (that is, Signed Data Format through pad patterns), followed by the static data to be authenticated as specified in section 7.3.1 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification*. If the Static Data Authentication Tag List is present and contains tags other than '82', then SDA has failed.
6. Use the designated hash algorithm (obtained from hash algorithm identifier) in the connecting results of the previous step to produce the hash result.
7. Compare the calculated hash result from the previous step with the recovered hash Result. If they are not the same, SDA has failed.
8. If all of the above steps were executed successfully, SDA was successful. The Data Authentication Code recovered in Table 7 shall be stored in tag '9F45'.

### 3.3 Dynamic Data Authentication(DDA)

The purpose of DDA is to confirm the legality of the critical data stored in and generated by IC card and the data received from terminal. Besides executing the static data authentication process similar to SDA, ensuring that the issuer data in IC card have not been illegally tampered with since personalization, DDA can also prevent the possibility of falsifying such cards.

The following two optional modes of dynamic data authentication are available:

- Standard dynamic data authentication: This mode is performed before card behavior analysis. In this mode, IC card generates a digital signature according to data stored in or generated by IC card identified by IC card dynamic data

and the data received from terminal identified by dynamic data authentication data object list.

- Combined dynamic data authentication/Applied Cryptogram Generation: This mode is performed after command GENERATE AC is run. In case of transaction certificate or authorized request cryptogram, IC card obtains a digital signature according to the data stored in or generated by IC card identified by IC card dynamic data, the data includes transaction certificate or authorized request cryptogram, as well as the options supported by IC card and pointed out by an unpredictable data AIP and generated by terminal and identified by card risk management data object list (it is CDOL1 for first command GENERATE AC and CDOL2 for second command GENERATE AC).

IC cards supporting dynamic data authentication must include the following data elements:

- Public key index of certification authority: The single byte data element contains a binary number, indicating which of the corresponding certification authority public keys stored in terminal should be used to verify IC card.
- Issuer public key certificate: The variable data element is provided by corresponding certification authority to Issuer. If verifying data element, the terminal authenticates the issuer public key and other data according to the process described in Section 3.3.3.
- IC card public key certificate: This variable data element is provided by Issuer to IC card. If verifying this data element, the terminal authenticates the IC card public key and other data according to the process described in Section 3.3.4.
- Issuer Public Key Remainder: It is a variable data element and is further interpreted in Section 3.3.1.
- Issuer Public Key Exponent: It is a variable data element provided by Issuer and is further interpreted in Section 3.3.1.
- Remainder of IC card public key: It is a variable data element and is further interpreted in Section 3.3.1.
- IC card public key index: It is a variable data element provided by Issuer and is further interpreted in Section 3.3.1.
- IC card private key: It is a variable data element stored in IC card used for generating signed dynamic application data according to the process described in Section 3.3.5 and Section 3.3.6.

The IC cards supporting DDA must generate following data elements:

- Signed dynamic application data: It is a variable data element generated by IC card private key corresponding to the IC card public key authenticated by IC card public key certificate. It is a digital signature, containing critical data

elements stored in or generated by IC card, as well as the critical data elements in terminal described in Section 3.3.5 and Section 3.3.6.

In order to support dynamic data authentication, every terminal must be able to save six certification authority public keys for every registered application provider identifier, and must relate the key information to every key (so that the terminal is able to support multi-algorithm in the future and transit from one algorithm to another. Please refer to Section 7.3). In case of a given RID and that the IC card provides certification authority public key, the terminal must be able to locate such public key and the information related to public key.

DDA must use a reversible algorithm shown in Section 9.2.1 and Section 10.2. Section 3.3.1 contains overviews of key and certificate involved in DDA process, and the initial steps of the authentication process are described in detail in Section 3.3.2 and Section 3.3.4:

- Certification authority public key restored by terminal
- Issuer public key restored by terminal
- IC card public key restored by terminal

Finally, the generation and verification process of dynamic signature in two cases are described in detail in Section 3.3.5 and 3.3.6.



In order to support DDA, an IC card must have a unique public/private key pair of its own; the public/private key pair consists of a private signature key and a corresponding public verification key. The IC card public key must be stored in the public key certificate of the IC card.

DDA uses a three-tier public key certificate scheme. Every IC card public key is authenticated by its Issuer and the certification authority authenticates the issuer public key, which indicates that in order to verify the signature of IC card, the terminal should first recover and verify IC card public key through verifying two certificates and then uses the public key to verify dynamic signature of the IC card.

Apply certification authority private key  $S_{CA}$  to the data designated in Table 8 and issuer private key  $S_I$  to the data designated in Table 9 according to the signature scheme in Section 9.2.1 to obtain issuer public key certificate and IC card public key certificate respectively.

There is a  $N_{CA}$  bytes public key modulus of certification authority public key pair and the certification authority public key index must be equal to 3 or  $2^{16}+1$ .

The issuer public key pair has an  $N_I$  bytes ( $N_I \leq N_{CA}$ ) issuer public key modulus. If  $N_I > (N_{CA}-36)$ , the issuer public key modulus is divided into two parts, one part contains the top  $N_{CA}-36$  bytes of the modulus (the leftmost numbers of issuer public key) and the other contains the rest lowest  $N_I - (N_{CA}-36)$  bytes of the modulus (remainder of the issuer public key). The issuer public key exponent must be equal to 3 or  $2^{16}+1$ .

The IC card public key pair has an  $N_{IC}$  bytes ( $N_{IC} \leq N_I \leq N_{CA}$ ) IC card public key modulus. If  $N_{IC} > (N_I-42)$ , the IC card public key modulus is divided into two parts. One part contains the top  $N_I-42$  bytes of the modulus (the leftmost numbers of IC card public key) and the other contains the rest lowest  $N_{IC} - (N_I-42)$  bytes of the modulus (the remainder of IC card public key). The IC card public key index must be equal to 3 or  $2^{16}+1$ .

If the static application data in the card is not unique (for example, the card uses different CVMs for international and domestic transactions), the card must support multi IC card public key certificate, if the signed static application data may be modified after being issued by the card, the card must support update of the IC card public key certificate.

In order to complete DDA, the terminal must first recover and verify IC card public key (this step is called IC card public key authentication). All information and data required for IC card public key authentication are detailed in Table 10 and stored in the IC card. Besides RID can be obtained from AID, other information and data can be obtained via command READ RECORD. If any of these data is missing, then DDA has failed.

Table 8 Issuer Public Key Data Signed by Certification Authority (Hash Algorithm Input)

| Field Name         | Length | Description                                                                                           | Format |
|--------------------|--------|-------------------------------------------------------------------------------------------------------|--------|
| Certificate format | 1      | Hexadecimal value is '02'                                                                             | b      |
| Issuer identifier  | 4      | The leftmost 3-8 numerals of the primary account number (adding hexadecimal numeral 'F' in the right) | cn8    |
| Certificate        | 2      | MMYY, the certificate is invalid after                                                                | n4     |



|                                                             |                          |                                                                                                                                                                                                                                           |   |
|-------------------------------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| Expiration Date                                             |                          | that date                                                                                                                                                                                                                                 |   |
| Certificate Serial No.                                      | 3                        | The only binary number distributed by certification authority to the certificate                                                                                                                                                          | b |
| Identifier of Hash algorithm                                | 1                        | It is an identifier that is used in hash algorithm to generate hash result in digital signature scheme.                                                                                                                                   | b |
| Identifier of issuer public key algorithm                   | 1                        | It is an identifier for the digital signature algorithm to use issuer public key.                                                                                                                                                         | b |
| Issuer Public Key Length                                    | 1                        | It is used to indicate the length of bytes of issuer public key modulus.                                                                                                                                                                  | b |
| Issuer Public Key Exponent Length                           | 1                        | It is used to indicate the length of bytes of issuer public key exponent.                                                                                                                                                                 | b |
| Issuer public key or the leftmost byte of issuer public key | $N_{CA} - 36$            | If $N_I \leq N_{CA} - 36$ , the field contains the whole issuer public key with $N_{CA} - 36 - N_I$ bytes of value 'BB' added in the right. If $N_I > N_{CA} - 36$ , the field contains the top $N_{CA} - 36$ bytes of issuer public key. | b |
| Issuer Public Key Remainder                                 | 0 or $N_I - N_{CA} + 36$ | The field only appears If $N_I > N_{CA} - 36$ and it contains the lowest $N_I - N_{CA} + 36$ bytes of issuer public key.                                                                                                                  | b |
| Issuer Public Key Exponent                                  | 1 or 3                   | Issuer public key exponent is equal to 3 or $2^{16} + 1$ .                                                                                                                                                                                | b |

Table 9 IC Card Public Key Data Signed by the Issuer (i.e. Hash Algorithm Input)

| Field Name                            | Length | Description                                                          | Format |
|---------------------------------------|--------|----------------------------------------------------------------------|--------|
| Certificate format                    | 1      | Hexadecimal and the value is '04'                                    | b      |
| Primary account number of application | 10     | Primary account number (adding hexadecimal numeral 'F' in the right) | cn20   |
| Certificate Expiration Date           | 2      | MMYY, the certificate is invalid after that date                     | n4     |

|                                                           |                          |                                                                                                                                                                                                                                    |   |
|-----------------------------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| Certificate Serial No.                                    | 3                        | The only binary number distributed by certification authority to the certificate                                                                                                                                                   | b |
| Identifier of Hash algorithm                              | 1                        | It is an identifier that is used in Hash algorithm to generate hash result in digital signature scheme.                                                                                                                            | b |
| Identifier of IC card public key algorithm                | 1                        | It is an identifier for the digital signature algorithm used in IC card public key.                                                                                                                                                | b |
| Length of IC card public key                              | 1                        | It is used to identify byte length of the IC card public key modulus.                                                                                                                                                              | b |
| Length of IC card public key index                        | 1                        | It is used to identify byte length of the IC card public key index.                                                                                                                                                                | b |
| IC card public key or leftmost byte of IC card public key | $N_I - 42$               | If $N_{IC} \leq N_I - 42$ , the field contains the whole issuer public key with $N_I - 42 - N_{IC}$ bytes of value 'BB' added on right. If $N_{IC} > N_I - 42$ , the field contains the top $N_I - 42$ bytes of issuer public key. | b |
| Remainder of IC card public key                           | 0 or $N_{IC} - N_I + 42$ | The field only appears If $N_{IC} > N_I - 42$ and it contains the lowest $N_{IC} - N_I + 42$ bytes of IC card public key.                                                                                                          | b |
| IC card public key index                                  | 1 or 3                   | IC card public key index is equal to 3 or $2^{16} + 1$                                                                                                                                                                             | b |
| Static Data to be Authenticated                           | Variable                 | Static data to be authenticated are described in detail in Section 7.3.1 of <i>UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification</i> .                     | b |

The input of authentication process consists of records identified by AFL and is followed by AIP (if AIP is identified by optional SDA tag list (tag '9F4A')). If there is a SDA tag list, it must only contain tag '82' for identifying AIP.

Table 10 Data Objects Required for Public Key Authentication in Dynamic authentication

| Field Name | Length | Description | Format |
|------------|--------|-------------|--------|
|------------|--------|-------------|--------|

|        |                     |                                                                                                                                                                                                                     |   |
|--------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| -      | 5                   | Identifier of registered application provider                                                                                                                                                                       | b |
| '8F'   | 1                   | Certification authority public key index                                                                                                                                                                            | b |
| '90'   | $N_{CA}$            | Certification authority public key certificate                                                                                                                                                                      | b |
| '92'   | $N_I - N_{CA} + 36$ | Issuer Public Key Remainder (if there is one)                                                                                                                                                                       | b |
| '9F32' | 1 or 3              | Issuer Public Key Exponent                                                                                                                                                                                          | b |
| '9F46' | $N_I$               | IC card public key certificate                                                                                                                                                                                      | b |
| '9F48' | $N_{IC} - N_I + 42$ | Remainder of IC card public key (if any)                                                                                                                                                                            | b |
| '9F47' | 1 or 3              | IC card public key index                                                                                                                                                                                            | b |
| —      | Variable            | Static data requiring authentication are described in detail in Section 7.3.1 of <i>UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification</i> . | - |

### 3.3.2 Obtain Certification Authority Public Key

The terminal reads out certification authority public key index. By this index and RID, the terminal is able to confirm and obtain the modulus and index of certification authority public key and related key information as well as the corresponding algorithm that will be used and stored in the terminal. If the terminal has not stored the key related to the index and RID, the DDA has failed.

### 3.3.3 Obtain Issuer Public Key

1. If the length of issuer public key certificate is different from that of certification authority public key modulus obtained in the previous clause, the DDA has failed.
2. In order to obtain the restored data shown in Table 11, use certification authority public key and corresponding algorithm to resume issuer public key certificate according to the retrieval function shown in Section 9.2.1. If the end of restored data is not equal to 'BC', the DDA has failed.

Table 11 Format of Data Restored from Issuer Public Key Certificate

| Field Name | Length | Description | Format |
|------------|--------|-------------|--------|
|------------|--------|-------------|--------|

|                                                             |             |                                                                                                                                                                                                                             |     |
|-------------------------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Recovered data head                                         | 1           | Hexadecimal value is '6A'                                                                                                                                                                                                   | b   |
| Certification Format                                        | 1           | Hexadecimal value is '02'                                                                                                                                                                                                   | b   |
| Identifier of Issuer                                        | 4           | The leftmost 3-8 numerals of the primary account number (adding hexadecimal numeral 'F' in the right)                                                                                                                       | cn8 |
| Certificate Expiration Date                                 | 2           | MMYY, the certificate is invalid after that date                                                                                                                                                                            | n4  |
| Certificate Serial Number                                   | 3           | The only binary number distributed by certification authority to the certificate                                                                                                                                            | b   |
| Identifier of Hash algorithm                                | 1           | It is an identifier used in hash algorithm to generate hash result in digital signature scheme.                                                                                                                             | b   |
| Identifier of issuer public key algorithm                   | 1           | It is an identifier for the digital signature algorithm used in issuer public key.                                                                                                                                          | b   |
| Issuer Public Key Length                                    | 1           | It is used to identify issuer public key length in byte.                                                                                                                                                                    | b   |
| Issuer Public Key Exponent Length                           | 1           | It is used to identify issuer public key exponent length in byte.                                                                                                                                                           | b   |
| Issuer public key or the leftmost byte of issuer public key | $N_{CA}-36$ | If $N_I \leq N_{CA}-36$ , the field contains the whole issuer public key with $N_{CA}-36-N_I$ bytes of value 'BB' added on right. If $N_I > N_{CA}-36$ , the field contains the top $N_{CA}-36$ bytes of issuer public key. | b   |
| Hash result                                                 | 20          | Issuer public key and the hash values of relevant information                                                                                                                                                               | b   |
| Recovered data end                                          | 1           | Hexadecimal and the value is 'BC'                                                                                                                                                                                           | b   |

3. Check the recovered data head. If it is not '6A', the DDA has failed.

4. Check certificate format. If it is not '02', the DDA has failed.

5. Concatenate from left to right the second to the tenth data elements in Table 11 (that is, Certificate Format through Issuer Public Key or Leftmost Digits of the Issuer Public Key), followed by the Issuer Public Key Remainder (if present), and finally the Issuer Public Key Exponent.
6. Apply the indicated hash algorithm (derived from the Hash Algorithm Indicator) to the result of the concatenation of the previous step to produce the hash result.
7. Compare the calculated hash result from the previous step with the recovered hash result. If they are not the same, dynamic data authentication has failed.
8. Verify that the Issuer Identifier matches the leftmost 3-8 PAN digits (allowing for the possible padding of the Issuer Identifier with hexadecimal 'F's). If not, dynamic data authentication has failed.
9. Verify that the last day of the month specified in the Certificate Expiration Date is equal to or later than today's date. If the Certificate Expiration Date is earlier than today's date, the certificate has expired, in which case dynamic data authentication has failed.
10. Verify that the concatenation of RID, Certification Public Key Index, and Certificate Serial Number is valid. If not, dynamic data authentication has failed.
11. If the Issuer Public Key Algorithm Indicator is not recognized, dynamic data authentication has failed.
12. If all the checks above are correct, concatenate the Leftmost Digits of the Issuer Public Key and the Issuer Public Key Remainder (if present) to obtain the Issuer Public Key Modulus, and continue with the next steps to obtain the IC Card Public Key.

### 3.3.4 Obtain IC Card Public Key

1. If the length of IC card public key certificate is different from that of issuer public key modulus obtained in the previous chapter, the DDA has failed.
2. In order to obtain the recovered data specified in Table 12, use issuer public key and the corresponding algorithm to apply the recovery function shown in Section 9.2.1 to IC card public key certificate. If the end of restored data is not equal to 'BC', the DDA has failed.

Table 12 Format of Data Recovered from IC Card Public Key Certificate

| Field Name           | Length | Description                       | Format |
|----------------------|--------|-----------------------------------|--------|
| Recovered data head  | 1      | Hexadecimal and the value is '6A' | b      |
| Certification format | 1      | Hexadecimal and the value is '04' | b      |

|                                                               |          |                                                                                                                                                                                                                            |      |
|---------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Primary account number of application                         | 10       | Primary account number (adding hexadecimal numeral 'F' in the right)                                                                                                                                                       | cn20 |
| Certificate Expiration Date                                   | 2        | MMYY, the certificate is invalid after that date                                                                                                                                                                           | n4   |
| Certificate Serial No.                                        | 3        | The only binary number distributed by issuer to the certificate                                                                                                                                                            | b    |
| Identifier of Hash algorithm                                  | 1        | It is an identifier used in hash algorithm to generate hash result in digital signature scheme.                                                                                                                            | b    |
| Identifier of IC card public key algorithm                    | 1        | It is an identifier for the digital signature algorithm used in IC card public key.                                                                                                                                        | b    |
| Length of IC card public key                                  | 1        | It is used to identify the byte length of IC card public key modulus.                                                                                                                                                      | b    |
| Length of IC card public key index                            | 1        | It is used to identify the byte length of IC card public key index.                                                                                                                                                        | b    |
| IC card public key or the leftmost byte of IC card public key | $N_I-42$ | If $N_{IC} \leq N_I-42$ , the field contains the whole IC card public key with $N_I-42-N_{IC}$ bytes of value 'BB' added on right. If $N_{IC} > N_I-42$ , the field contains the top $N_I-42$ bytes of IC card public key. | b    |
| Hash result                                                   | 20       | IC card public key and the hash value of relevant information                                                                                                                                                              | b    |
| Recovered data end                                            | 1        | Hexadecimal and the value is 'BC'                                                                                                                                                                                          | b    |

3. Check recovered data head. If it is not '6A', the DDA has failed.

4. Check certificate format. If it is not '04', the DDA has failed.

5. Concatenate from left to right the second to the tenth data elements in Table 13 (that is, Certificate Format through ICC Public Key or Leftmost Digits of the IC Card Public Key), followed by the ICC Public Key Remainder (if present), the ICC Public Key Exponent, and finally the static data to be authenticated specified in section 7.3.1 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification*. If the Static Data Authentication Tag List is present and contains tags other than '82', then dynamic data authentication has failed.

6. Apply the indicated hash algorithm (derived from the Hash Algorithm Indicator) to the result of the concatenation of the previous step to produce the hash result.
7. Compare the calculated hash result from the previous step with the recovered hash result. If they are not the same, dynamic data authentication has failed.
8. Compare the recovered PAN to the Application PAN read from the IC Card. If they are not the same, dynamic data authentication has failed.
9. Verify that the last day of the month specified in the Certificate Expiration Date is equal to or later than today's date. If not, dynamic data authentication has failed.
10. If the IC Card Public Key Algorithm Indicator is not recognized, dynamic data authentication has failed.
11. If all of the above checks are correct, concatenate the Leftmost Digits of the IC Card Public Key and the IC Card Public Key Remainder (if present) to obtain the IC Card Public Key Modulus, and continue with the actual dynamic data authentication described in the two sections below.

### 3.3.5 Standard DDA

#### 3.3.5.1 Generation of Dynamic Signature

Suppose that the terminal has obtained IC card public key according to the procedures described above, generation of dynamic signature should be conducted according to the following steps:

1. The terminal issues a command of internal authentication (INTERNAL AUTHENTICATE), which includes the data elements designated by DDOL and the data elements are connected together according to the regulations in Section 3.3 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification*.

The IC card may contain DDOL but the terminal should have a default DDOL designated by the payment system to prevent the IC card from being used when DDOL is not provided.

DDOL must include the unpredictable number generated beforehand by terminal (tag '9F37', a 4-byte binary number).

If any of the following situations occurs, the DDA has failed.

- Both IC card and terminal contain no DDOL.
  - The DDOL in the IC Card does not include the Unpredictable Number.
  - The IC Card does not contain a DDOL and the default DDOL in the terminal does not include the Unpredictable Number.
2. IC card generates digital signature using IC card private key and the corresponding algorithm according to the data in Table 13 in Section 9.2.1. The result is called signed dynamic application data.

Table 13 Dynamic Application Data Requiring Signature (i.e. Hash Algorithm Input)

| Field Name                     | Length                               | Description                                                         | Format |
|--------------------------------|--------------------------------------|---------------------------------------------------------------------|--------|
| Signed data format             | 1                                    | Hexadecimal and the value is '05' or '95'                           | b      |
| Identifier of Hash algorithm   | 1                                    | It is used to identify the hash algorithm generating hash result.   | b      |
| Length of IC card dynamic data | 1                                    | It is used to identify the byte length LDD of IC card dynamic data. | b      |
| IC card dynamic data           | L <sub>DD</sub>                      | Dynamic data generated by IC card and/or stored on IC card          | -      |
| Pad patterns                   | N <sub>IC</sub> -L <sub>DD</sub> -25 | (N <sub>IC</sub> -L <sub>DD</sub> -25)padding bytes with value 'BB' | b      |
| Terminal dynamic data          | Variable                             | It is formed via connecting the data elements designated by DDOL.   | -      |

- For qUICS online authorization requests (ARQC), the Signed Data Format input to the DDA hash algorithm must be the value '95' for ODA for online authorization. The Signed Data Format of '95' distinguishes the dynamic signature returned for an online authorization request from the dynamic signature returned for an offline authorization request (where Signed Data Format is the value '05').
- For offline approval requests (TC), the Signed Data Format input to the DDA hash algorithm must be the value '05'.
- If a contactless-enabled terminal supports ODA for online authorization, the terminal should support Signed Data Format '95' in dynamic signature verification for ODA for online authorization. In a traditional financial POS environment which does not support ODA for online authorization, the DDA will fail when the Signed Data Format is not '05'.

*Note: For the mobile payment product based on host-based card emulation (HCE) technology, in addition to the above-mentioned requirement of Signed Data Format, the IC Card Public Key Certificate Format must be '94' when it is used in an ODA for online authorization. For the SE-based mobile payment product and traditional plastic IC card, the IC Card Public Key Certificate Format is still '04'. The*



terminal that supports ODA for online authorization must support both values ('04' and '94').

The byte length  $L_{DD}$  of IC card dynamic data should meet the condition  $0 \leq L_{DD} \leq N_{IC}-25$ . The leftmost 3-9 bytes of IC card dynamic data should consist of a one byte ICC dynamic data followed by 2-8 values of IC card dynamic data (tag '9F4C', 2-8 binary bytes). IC card dynamic data is a parameter generated by IC card and varying with the time (for example, it can be an unpredictable number or a counter which adds 1 every time when the IC card receives a command of internal authentication (INTERNAL AUTHENTICATE)). *UnionPay Integrated circuit card specifications-Basic Specification* using ATC as IC card dynamic data.

In addition to the data specified in Table 10, the data objects required for DDA are detailed in Table 14.

Table 14 Other Data Objects Required for Generating and Verifying Dynamic Signature

| Field Name | Length   | Value                           | Format |
|------------|----------|---------------------------------|--------|
| '9F4B'     | $N_{IC}$ | Signed dynamic application data | b      |
| '9F49'     | Variable | DDOL                            | b      |

### 3.3.5.2 Verification of Dynamic Signature

1. If the length of signed dynamic application data is different from that of IC card public key modulus, the DDA has failed.
2. In order to obtain the recovered data shown in Table 15, use IC card public key and corresponding algorithm to apply the recovery function shown in Section 9.2.1 to signed dynamic application data. If the end of restored data is not equal to 'BC', the DDA has failed.

Table 15 Format of Data Restored from Signed Dynamic Application Data

| Field Name                   | Length | Description                                                                                      | Format |
|------------------------------|--------|--------------------------------------------------------------------------------------------------|--------|
| Recovered data head          | 1      | Hexadecimal and the value is '6A'                                                                | b      |
| Signed data format           | 1      | Hexadecimal and the value is '05' or '95'                                                        | b      |
| Identifier of Hash algorithm | 1      | It is an identifier used in hash algorithm1 to generate hash result in digital signature scheme. | b      |

|                                |                    |                                                                         |   |
|--------------------------------|--------------------|-------------------------------------------------------------------------|---|
| Length of IC card dynamic data | 1                  | It is used to identify the byte length of IC card dynamic data.         | b |
| IC card dynamic data           | $L_{DD}$           | Dynamic data generated by IC card and/or stored on IC card              | - |
| Pad patterns                   | $N_{IC}-L_{DD}-25$ | $(N_{IC}-L_{DD}-25)$ padding bytes with value 'BB'                      | b |
| Hash result                    | 20                 | Dynamic application data and the hash value of the relevant information | b |
| Recovered data end             | 1                  | Hexadecimal and the value is 'BC'                                       | b |

3. Check recovered data head and if it is not '6A', the DDA has failed.
4. Check the signed data format. For terminals which support ODA for online authorization, the terminals shall use signed data format of the value '95' to verify data during the dynamic signature verification process of ODA for online authorization. For terminals which do not support ODA for online authorization or which support ODA for online authorization but do not use ODA for online authorization in certain transactions, the terminals shall use signed data format of the value '05' to verify data during the dynamic signature verification process.. If the signed data format is incorrect, the DDA has failed.
5. Concatenate from left to right the second to the sixth data elements in Table 15 (that is, Signed Data Format until Pad patterns), followed by the data elements specified by the DDOL.
6. Apply the indicated hash algorithm (derived from the Hash Algorithm Indicator) to the result of the concatenation of the previous step to produce the hash result.
7. Compare the calculated hash result from the previous step with the recovered hash result. If they are not the same, DDA has failed.

If all the above steps were executed successfully, DDA was successful. The ICC Dynamic Number contained in the ICC Dynamic Data recovered in Table 15 should be stored in tag '9F4C'.

### 3.3.6 Combined DDA/Application Cryptogram Generation (CDA)

CDA consists of a dynamic signature generated by the IC followed by verification of the signature by the terminal. During retrieval of the public keys, errors may result in CDA failure (TVR bit for 'CDA failed' is set to 1). These errors include but are not limited to failure of public key retrieval and invalid format of records to be authenticated.

For the first GENERATE AC command, and for the second GENERATE AC command in the case “unable to go online”, the cryptogram type requested by the terminal is always determined by the final Terminal Action Analysis preceding the GENERATE AC command. If any of the above errors are detected prior to the final Terminal Action Analysis, then the terminal shall not request CDA in the GENERATE AC command. When the GENERATE AC command is issued with a CDA request, then if any of the above errors are detected subsequently, the eventual result will be an offline transaction decline.

For sections 3.3.6.1 and 3.3.6.2, assuming that the following conditions are met:

- Both the IC card and the terminal support CDA/AAC.
- The cryptogram to be requested is not an Application Authentication Cryptogram (AAC), i.e. Terminal Action Analysis has not resulted in offline decline.
- The TVR bit for “CDA failed” is not set to 1 prior to final Terminal Action Analysis. Except when returning an AAC, the IC card always replies with a CDA signature when requested by the terminal.

In the case of the first GENERATE AC command:

- When requesting an ARQC, the terminal may request it with or without a CDA signature. When an ARQC is requested without a CDA signature, then the terminal shall set the TVR bit for “Offline data authentication was not performed” to 1 prior to issuance of the GENERATE AC command. When an ARQC is requested without a CDA signature, the processes described in sections 3.3.6.1 and 3.3.6.2 are not performed.
- When requesting a TC, the terminal shall request it with a CDA signature.
- When requesting an AAC, the terminal shall request it without a CDA signature.

In the case of the second GENERATE AC command:

- The terminal shall set the TVR bit for “Offline data authentication was not performed” to 0 prior to issuance of the GENERATE AC command. If the terminal is processing the transaction as “unable to go online” then the TVR bit setting will be done before the associated terminal action analysis.
- When requesting a TC:
  - If the terminal is processing the transaction as “unable to go online” (and the result of terminal action analysis is to request a TC), then the terminal shall request a TC with a CDA signature.
  - If the terminal is not processing the transaction as “unable to go online”, then the terminal may request the TC with or without a CDA signature.
- When requesting an AAC, the terminal requests it without a CDA signature.

### 3.3.6.1 Dynamic signature Generation

CDA and application cryptogram generation should be performed according to the following steps:

1. Terminal issues an application cryptogram generation command (GENERATE AC) according to definitions in appendix B. 6 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification*, with CDOL request in position 1.

2. If IC card uses TC or ARQC as the response, the IC card adopts the following steps:

- 1) IC card generates TC or ARQC.
- 2) IC card uses the hash algorithm indicated by Hash algorithm identifier to calculate the following data elements connected from left to right:
  - In the case of the first command GENERATE AC:
    - ✓ As shown in PDOL and in the sequence as appears in PDOL, the value of data element the terminal transmits to IC card with command GET PROCESSING OPTIONS.
    - ✓ As shown in PDOL1 and in the sequence as appears in PDOL1, the value of data element the terminal transmits to IC card with GENERATE AC command.
    - ✓ The IC card responses to GENERATE AC command with tag, length and value in the returned data element, in order, with the signed dynamic application data excluded.
  - In the case of the second GENERATE AC command:
    - ✓ As shown in PDOL and in the sequence as appears in PDOL, the value of data element the terminal transmits to IC card with GET PROCESSING OPTIONS command.
    - ✓ As shown in PDOL1 and in the sequence as appears in PDOL1, the value of data element the terminal transmits to IC card with the first GENERATE AC command.
    - ✓ As shown in PDOL2 and in the sequence as appears in PDOL2, the value of data element the terminal transmits to IC card with the second GENERATE AC command.
    - ✓ The IC card responses to GENERATE AC command with tag, length and value in the returned data element, in order, with the signed dynamic application data excluded. The 20 bytes calculation result is called the transaction data hash value.
- 3) IC card uses the IC card secret key  $S_{IC}$  stored in card to calculate the data in Table 16 and the digital signature scheme and corresponding algorithm are

defined in Section 9.2.1. The result is called the signed dynamic application data.

Table 16 Dynamic Application Data to be signed (Hash Algorithm Input)

| Field Name                     | Length             | Description                                                                                                                       | Format |
|--------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------|
| Signed data format             | 1                  | Hexadecimal and the value is '05'                                                                                                 | b      |
| Identifier of Hash algorithm   | 1                  | It is used to identify the hash algorithm generating transaction data hash value and the hash result in digital signature scheme. | b      |
| Length of IC card dynamic data | 1                  | It is used to identify the byte length LDD of IC card dynamic data.                                                               | b      |
| IC card dynamic data           | $L_{DD}$           | Dynamic data generated by IC card and/or stored on IC card                                                                        | -      |
| Pad pattern                    | $N_{IC}-L_{DD}-25$ | $(N_{IC}-L_{DD}-25)$ padding bytes with a value of 'BB'                                                                           | b      |
| Unpredictable numbers          | 4                  | Unpredictable numbers generated by terminal                                                                                       | b      |

The byte length LDD of IC card dynamic data should satisfy the condition  $0 \leq L_{DD} \leq N_{IC}-25$ . The leftmost 32-38 bytes of IC card dynamic data consist of the concatenation of the data specified in Table 17.

Table 17 Contents of IC Card Dynamic Data

| Length | Value                          | Format |
|--------|--------------------------------|--------|
| 1      | Length of IC card dynamic data | b      |
| 2-8    | IC card dynamic data           | b      |
| 1      | Cryptogram information data    | b      |
| 8      | TC or ARQC                     | b      |
| 20     | Transaction data hash value    | b      |

IC card dynamic data is a parameter generated by IC card and varying with time. (For example, it can be an unpredictable numbers or a counter that increases by 1

every time the IC card receives a command of application cryptogram generation (GENERATE AC) while conducting transactions.) *UnionPay Integrated circuit card specifications-Basic Specification* recommends using ATC as IC card dynamic data.

Response of the IC card to command of application cryptogram generation (GENERATE AC) must be coded according to format 2 (structure data object with tag '77') as shown in Section GENERATE AC command response message data field format of Appendix B. 6 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* and it must include the three mandatory data objects shown in Table 18 (according to TLV coded in response) and optionally include the issuer application data.

Table 18 Data Objects Returned in Command GENERATE AC in CDA

| Tag    | Length                   | Value                           | Existence |
|--------|--------------------------|---------------------------------|-----------|
| '9F27' | 1                        | Cryptogram information data     | Mandatory |
| '9F36' | 2                        | Application transaction counter | Mandatory |
| '9F4B' | N <sub>IC</sub>          | Signed dynamic application data | Mandatory |
| '9F10' | Variable, max. length 32 | Issuer application data         | Optional  |

3. If IC card adopts AAC as the response, the response of IC card must be coded according to the format 1 or format 2 shown in Section GENERATE AC command response message data field format of Appendix B. 6 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* and it must include the three mandatory data objects shown in Table 19 and optionally include issuer application data.

Table 19 Data Objects Returned in Command GENERATE AC to Generate AAC

| Tag    | Length | Value                                 | Existence |
|--------|--------|---------------------------------------|-----------|
| '9F27' | 1      | Cryptogram information data           | Mandatory |
| '9F36' | 2      | Application transaction counter       | Mandatory |
| '9F26' | 8      | Application authentication cryptogram | Mandatory |

|        |                             |                         |          |
|--------|-----------------------------|-------------------------|----------|
| '9F10' | Variable, max.<br>length 32 | Issuer application data | Optional |
|--------|-----------------------------|-------------------------|----------|

Besides the cryptogram information data in plaintext, in the process of verifying signed dynamic application data, cryptogram information data can also be restored. If there are such data, the restored cryptogram information data must be used to determine the returned cryptogram type; otherwise, the cryptogram information data of plaintext should be used.

If there is issuer application data (tag '9F10'), encoding should be conducted according to the format shown in Table 20.

Table 20 Issuer Application Data

| Tag | Length   | Value                         | Existence |
|-----|----------|-------------------------------|-----------|
|     | 1        | Indicator of length           | Mandatory |
|     | 1        | Index of decentralized Key    | Mandatory |
|     | 1        | Edition number of cryptogram  | Mandatory |
|     | 4        | Card verification result(CVR) | Mandatory |
|     | 1        | Algorithm identifier          | Mandatory |
|     | Variable | Self-defined data of Issuer   | Optional  |

The decentralized Key index indicates which key is adopted by IC card to generate application cryptogram and the edition number of cryptogram indicates the algorithm of the application cryptogram. Section 6.1 describes the method for generating application cryptogram. Please refer to Appendix E of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* for definitions of cryptogram edition number and algorithm identifier.

### 3.3.6.2 Verification of dynamic signature

Suppose that the terminal has successfully retrieved IC card public key according to the process described above:

If the IC card makes responses with an AAC, the terminal should decline transaction.

If IC card responds with TC or ARQC, the terminal retrieves the first four data objects in Table 18 from the response and executes the following steps:

1. If the length of signed dynamic application data is different from that of IC card public key modulus, the combined DDA/application cryptogram generation (CDA) has failed.
2. In order to obtain the recovered data specified in Table 21, use IC card public key and corresponding algorithm to apply the recovery function specified in Section 9.2.1 to signed dynamic application data. If the end of restored data is not equal to 'BC', the combined DDA/application cryptogram generation (CDA) has failed.

Table 21 Format of Data Recovered from Signed Dynamic Application Data

| Field Name                     | Length                 | Description                                                                                                                   | Format |
|--------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------|
| Recovered data head            | 1                      | Hexadecimal and the value is '6A'                                                                                             | b      |
| Signed data format             | 1                      | Hexadecimal and the value is '05'                                                                                             | b      |
| Identifier of Hash algorithm   | 1                      | It is used to identify the hash algorithm generating transaction data hash value and digital signature scheme in hash result. | b      |
| Length of IC card dynamic data | 1                      | It is used to identify the byte length of IC card dynamic data.                                                               | b      |
| IC card dynamic data           | $L_{DD}$               | Dynamic data generated by IC card and/or stored on IC card                                                                    | -      |
| Pad patterns                   | $N_{IC} - L_{DD} - 25$ | $(N_{IC} - L_{DD} - 25)$ padding bytes with a value of 'BB'                                                                   | b      |
| Hash result                    | 20                     | Dynamic application data and the hash value of relevant information                                                           | b      |
| Recovered data end             | 1                      | Hexadecimal and the value is 'BC'                                                                                             | b      |

3. Check recovered data head and if it is not '6A', the combined dynamic data authentication/application cryptogram generation has failed.
4. Check signed data format and if it is not '05', the combined dynamic data authentication/application cryptogram generation has failed.
5. Retrieve the data specified in Table 17 from the IC card dynamic data.
6. Check that the Cryptogram Information Data retrieved from the IC Card Dynamic Data is equal to the Cryptogram Information Data obtained from the



response to the GENERATE AC command. If this is not the case, the combined DDA/application cryptogram generation has failed.

7. Concatenate from left to right the second to the sixth data elements in Table 21 (that is, Signed Data Format through Pad patterns), followed by the Unpredictable Number.

8. Apply the indicated hash algorithm (derived from the Hash Algorithm Indicator) to the result of the concatenation of the previous step to produce the hash result.

9. Compare the calculated hash result from the previous step with the recovered hash result. If they are not the same, the combined DDA/application cryptogram generation has failed

10. Concatenate from left to right the values of the following data elements:

- In the case of the first GENERATE AC command:
  - ✓ As shown in PDOL and in the sequence indicated in PDOL, the value of data element transmitted by the terminal to IC card in GET PROCESSING OPTIONS command.
  - ✓ As shown in CDOL1 and in the sequence indicated in CDOL1, the value of data element transmitted by the terminal to IC card in the first GENERATE AC command.
  - ✓ The IC card responses to the GENERATE AC command, with tag, length and value in the returned data element, in order, and the signed dynamic application data are excluded.
- In the case of the second GENERATE AC command:
  - ✓ As shown in PDOL and in the sequence indicated in PDOL, the value of data element transmitted by the terminal to IC card in GET PROCESSING OPTIONS command.
  - ✓ As is shown in CDOL1, it is the value of the data element transmitted to IC card by the terminal in the first GENERATE AC command in the order present in PDOL.
  - ✓ As is shown in CDOL2, it is the value of the data element transmitted to IC card by the terminal in the second GENERATE AC command in the order present in PDOL.
  - ✓ The IC card responses to the GENERATE AC command, with tag, length and value in the returned data element, in order, and the signed dynamic application data are excluded.

11. Apply the designated hash algorithm (obtained from Hash algorithm identifier) to the concatenated result of the last step so as to obtain transaction data hash value.

12. Compare the transaction data hash value obtained from the last step calculation with the transaction data hash value restored from IC card dynamic data in step 5. If

they are different, the combined DDA/application cryptogram generation (CDA) has failed.

If all above steps are successful, the combined DDA/application cryptogram generation succeeds. The IC card dynamic data and ARQC or TC contained in IC card dynamic data restored from Table 17 are accordingly stored in tag '9F4C' and tag '9F26'.

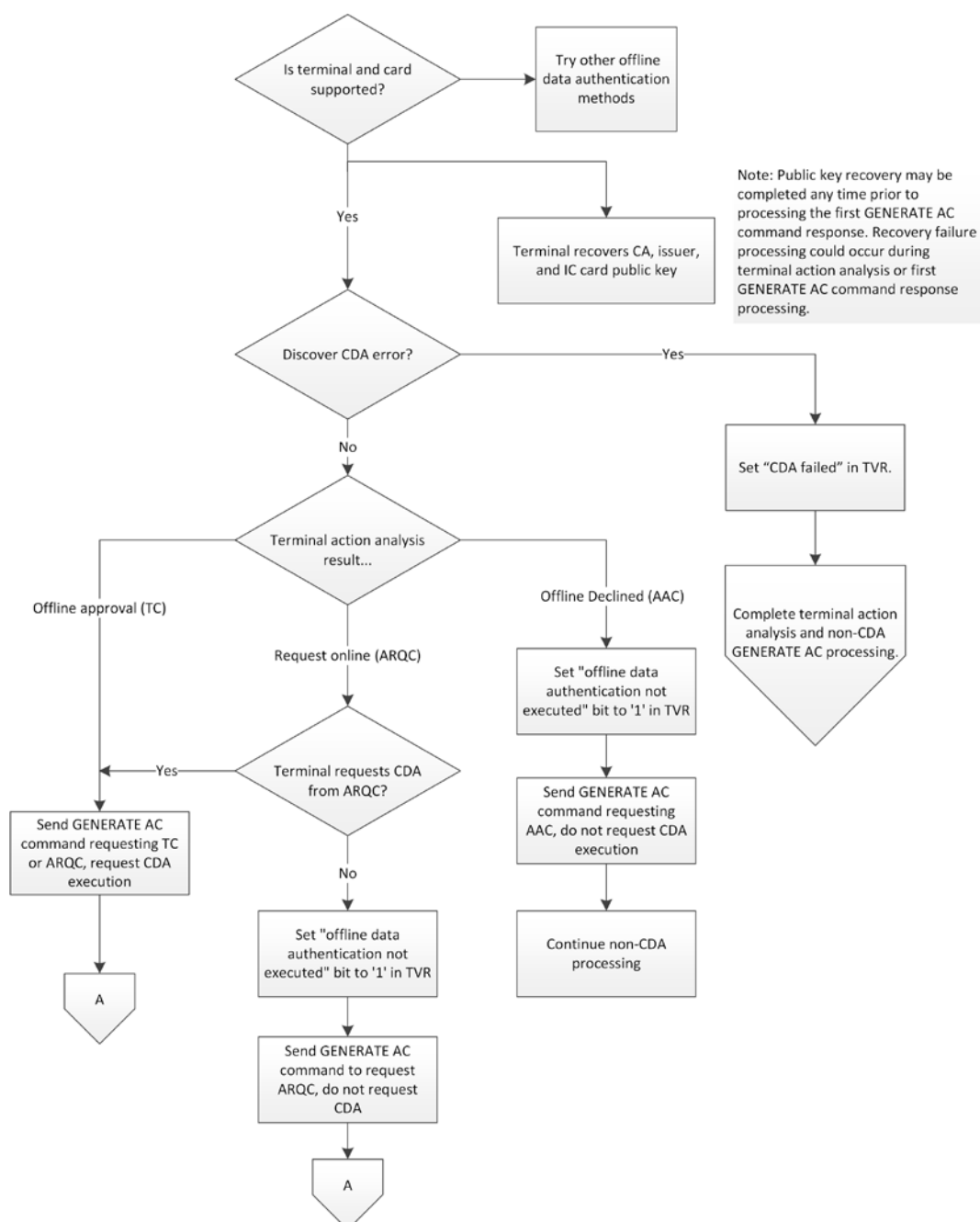


Figure 3 CDA Processing flow – Offline Data Authentication Processing

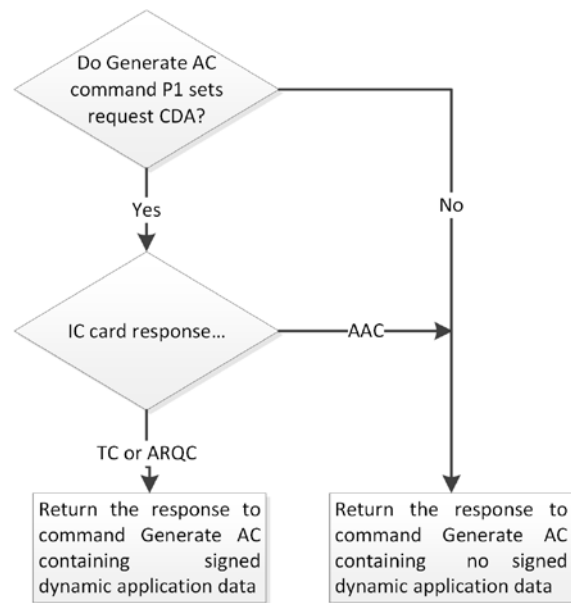


Figure 4 CDA Processing Flow – Card Generate AC Command Processing

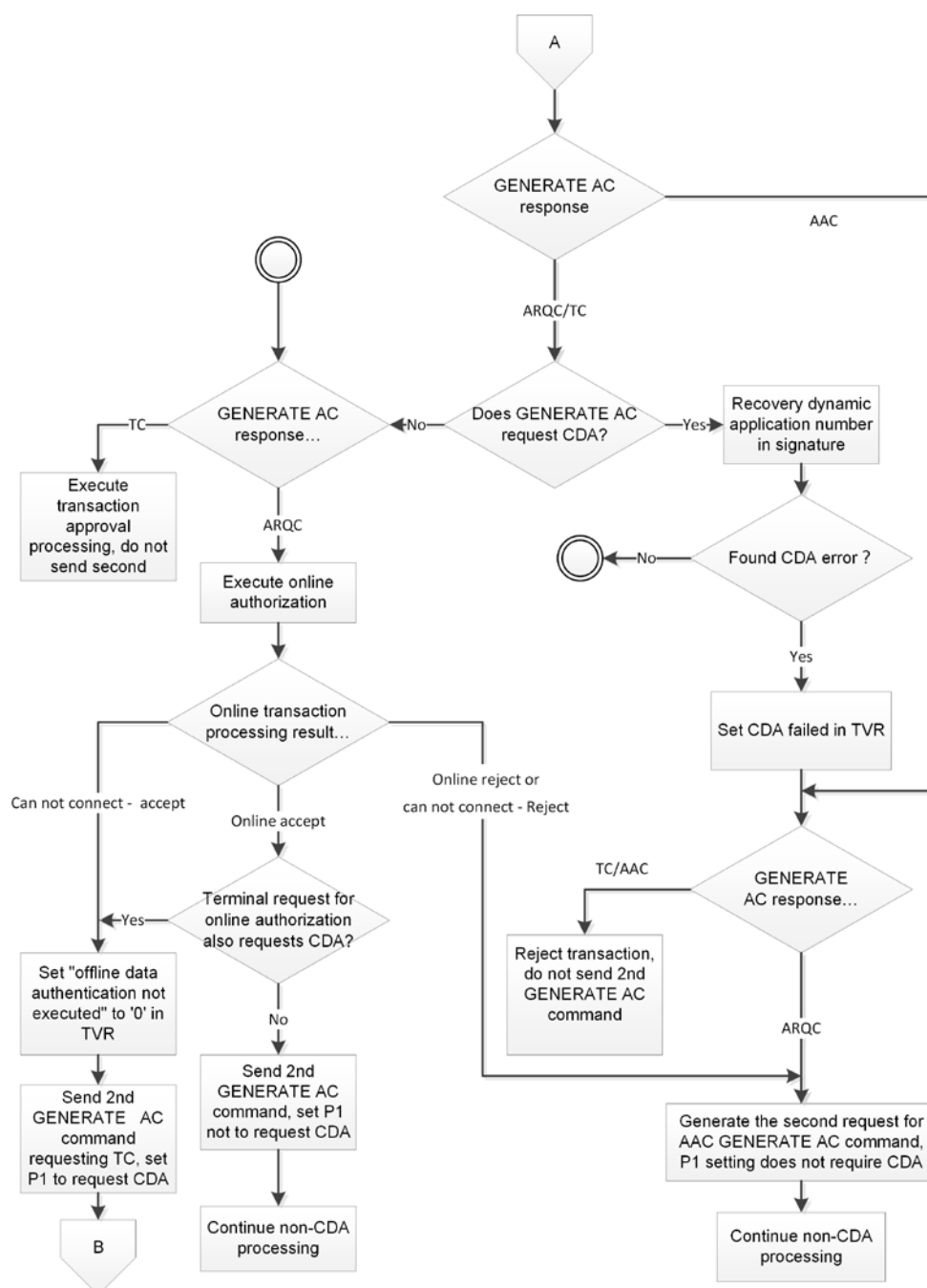


Figure 5 CDA Processing Flow – First Generate AC Command Response Processing

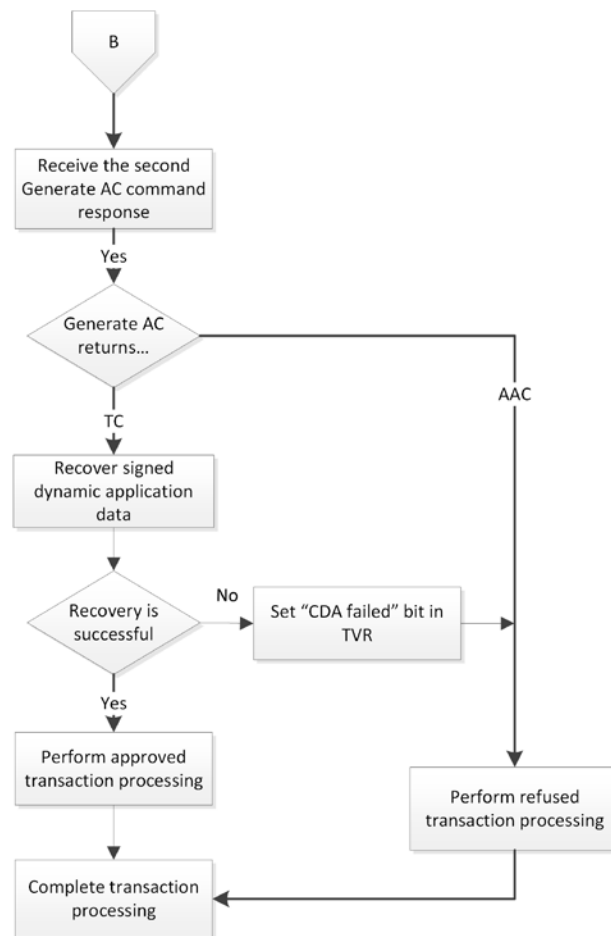


Figure 6 CDA Processing Flow – the Second Generate AC Command Response Processing

## 4 Application cryptogram and issuer authentication

This chapter describes the method for IC card to generate application cryptogram (TC, ARQC or AAC) and the method for Issuer to generate authorization response cryptogram (ARPC) verified by IC card. Please refer to *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* for more details on the role of these cryptograms in a transaction.

### 4.1 Application Cryptogram Generation

#### 4.1.1 Selection of data source

An Application Cryptogram consists of a Message Authentication Code (MAC) generated from the following data:

- Data in the IC card's DOL transmitted from the terminal to the ICC via the GENERATE AC or other command, and
- Data internally accessed in IC card

Please refer to Appendix D.1 of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* for the selection of data sources contained in generation of application cryptogram. The recommended minimum data set is described in detail in Table 22.

Table 22 Recommended Minimum Data Set Used in Application Cryptogram Generation

| Value                        | Source   |
|------------------------------|----------|
| Authorized amount (numeric)  | Terminal |
| Other amounts (numeric)      | Terminal |
| Terminal country code        | Terminal |
| Terminal verification result | Terminal |
| Transaction currency code    | Terminal |
| Transaction date             | Terminal |
| Transaction type             | Terminal |
| Unpredictable numbers        | Terminal |

|                                 |         |
|---------------------------------|---------|
| Application interchange profile | IC card |
| Application transaction counter | IC card |

Please see Table 23 for optional application cryptogram generation data source.

Table 23 Optional Application Cryptogram Generation Data Source

| Value                    | Source  |
|--------------------------|---------|
| Card verification result | IC card |

#### 4.1.2 Application Cryptogram Algorithm

Application cryptogram generation method uses a unique 16-bytes IC card application cryptogram (AC) key  $MK_{AC}$  and the data selected according to descriptions in Section 4.1.1 as the input, and then the 8-byte application cryptogram is calculated according to the following two steps:

1. In the first step, the IC card application cryptogram (AC) Unique Key  $MK_{AC}$  and the 2-bytes IC card application transaction counter are used as input and the 16-bytes application cryptogram process Key  $SK_{AC}$  is obtained via derivation, the process Key derivation function shown in Section 9.1.3 is used.
2. In the second step, the 16-bytes application cryptogram process key obtained via derivation in the last step is used and the MAC algorithm shown in Section 9.1.2 is applied to the selected data to generate 8-byte application cryptogram.

#### 4.2 Issuer Authentication

The method of generating 8-byte authorization response cryptogram is to encrypt the 8-byte long ARQC generated by IC card according to the method described in Section 4.1 and the 2-bytes authorization response code ARC via the 16-bytes application cryptogram process Key  $SK_{AC}$  (please refer to Section 4.1) according to the symmetrical encryption algorithm shown in Section 10.1:

1. One 8-byte number is obtained via adding 6 '00' bytes at the end of 2-bytes ARC.

$X := (ARC \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$ .

2. Calculating  $Y := ARQC \oplus X$ .

3. Calculating ARPC

8-bytes ARPC is obtained based on 64-bit blocked encryption algorithm.

$ARPC := ALG(SK_{AC})[Y]$

16-bytes ARPC is obtained based on 128-bit block encryption algorithm.

$$\text{ARPC} := \text{ALG}(\text{SK}_{\text{AC}}) [\text{Y} \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00']$$

### 4.3 Key Management

The mechanisms for Application Cryptogram and Issuer Authentication require the management by the Issuer of the unique IC Card Application Cryptogram Master Keys. Please refer to Section 9.1.4 for derivation method of the IC card application cryptogram (AC) sub-key.



## 5 Security Message

Security message ensures the integrity of data and authentication of the Issuer via message authentication code (MAC), and ensures the confidentiality of the data by encrypting the data field.

### 5.1 Message Format

Please refer to the definitions of *UnionPay Integrated circuit card specifications-Basic Specifications Part Two Debit/Credit Application Card Specification* for the message format used in *UnionPay Integrated circuit card specifications-Basic Specifications*.

The data field of the command involved in the message does not use BER-TLV encoding in security message. The transmitter using the command of security message and the current selected application must know that the data objects included in the data field and the lengths of those data objects. According to ISO/IEC 7816-4:2005, the security messages consistent with this format are clearly designated via setting the lower-nibble of the type-byte of the command as '4'.

### 5.2 Secure Message Integrity and Authentication

#### 5.2.1 Command Data Field

The transmitter using the command of security message and the current selected application must know the data elements included in the data field (including MAC) and the corresponding lengths of the data. MAC is not BER-TLV encoding and is always the last data element in the data field. The length of MAC is always 4-bytes.

Table 24 Command Data Field Format of Security Message for Integrity and Authentication

| Value 1               | Value 2       |
|-----------------------|---------------|
| Command data (if any) | MAC (4-bytes) |

#### 5.2.2 Derivation of MAC process Key

The first step of MAC generation of security message with integrity and authentication as the purpose includes obtaining a unique 16-bytes message authentication code (MAC) from the unique 16-bytes message authentication code (MAC) sub-key of IC card and 2-bytes ATC derivation. One type of derivation method is detailed in Section 9.1.3.

#### 5.2.3 MAC Calculation

MAC is obtained by using MAC process Key obtained via derivation by the method described in Section 5.2.2 and calculated through applying the system described in Section 9.1.2 to the protected message.

The message to be protected must be constructed according to special specifications of the payment system; however, it always includes the head of command APDU (CLA INS P1 P2) and the command data (If any).

The length of MAC in this specification is 4. After the 8-bytes result is obtained from the calculation method described above, the leftmost (the top) 4 bytes of the result are used to obtain MAC.

### 5.3 Message Confidentiality

#### 5.3.1 Command Data Field

Except MAC, all the other plaintext data fields in command data field are encrypted.

Table 25 Command Data Field Format of Security Message for Confidentiality

| Value 1                    | Value 2         |
|----------------------------|-----------------|
| Cryptogram(encrypted data) | MAC( If exists) |

#### 5.3.2 Encrypting Process Key Derivation

The first step of encryption and decryption of security message with private security as the purpose includes the unique 16-bytes encrypting process key obtained from the unique 16-bytes security message sub-key of IC card and 2-bytes ATC derivation. The method is detailed in Section 9.1.3.

#### 5.3.3 Encryption and Decryption

The encryption and decryption of plaintext/encryption command data field is conducted by using the encryption process key obtained via derivation according to the method described in Section 5.3.2 and applies the system described in Section 9.1.1.

### 5.4 Key Management

The mechanism for message authentication requires the Issuer to manage a unique IC Card message authentication code (MAC). Please refer to Section 9.1.4 for derivation method of the IC card message authentication code (MAC) sub-key.

## 6 Card Security

### 6.1 Symbiotic Application

In order to solve the security problems in independently managing different applications in one card, each application should be placed in an independent ADF. That means a “fire wall” should be designed between applications to prevent illegal access across the application. Additionally, no application should conflict with the personalization requirement and application rules coexisting in the card.

### 6.2 Key Independence

Encryption/decryption keys for a special function (such as AC key) must not be used by other functions, including the keys stored in IC card and the keys for generating, deriving and transmitting those keys.

### 6.3 Internal Security System of Card

This section introduces the structure of internal security system of the card. For the processing flow controlled by card operating system and affecting all card data or execution code, the use of the internal security system is restricted.

#### 6.3.1 Internal Security Target of Card

The target of this security system is to ensure that the card operating system adopts an appropriate security system to provide security and integrity guarantee for all data and processing flow in card. This system is designed for accessing data files and using command and encryption algorithm.

#### 6.3.2 Internal Security Overview of Card

The basic structure of this security system includes two basic characteristics:

- Establishment of “security field”.
- Designated access conditions are adopted for the access of every EF.

##### 6.3.2.1 Security Field

As the operating system controls the access of all data and executable resources (data file, record, command and encrypted key as well as algorithm, it is possible to establish a security field, which is realized via running command SELECT and GET PROCESSING OPTIONS. Those commands are used to establish relevant information describing the security field and (at any time) to define the access scope of the designated data and executable resources.

The card operating system uses the information and realizes the access control of the data on file level. Therefore, the Issuer must seriously consider how to amalgamate the data objects and data elements into the file. Put it another way, the data that can be accessed on the same level can be amalgamated into one file with similar data; on the flip side, the data with different access conditions should not be amalgamated into the same file.

The command SELECT is processed so that card operating system information, which is application management data (AMD), can be accessed. AMD designates all data files, records and executable resources that can be accessed by follow-up instructions.

Application management data (which are submitted to the operating system after application is selected) determines the files and executable resources that the application is able to access. Command GET PROCESSING OPTIONS will make the operating system to change the state of security field so that it is possible to quote other files and records. File numbers and record numbers should be provided by the command in application file locator (Application File Locator).

As security field is established because of the execution of commands SELECT and GET PROCESSING OPTIONS, the Issuer is able to restrict the resources that can be accessed during transaction, including determining whether the resource should be incorporated into application management data and application file locator or not. The data files that are not accessed by application management data and application file locator can't be accessed. Commands or encryption algorithms that are not accessed by management data and application file locator should not be used in the current security field scope. Initialized state of application management data (defined during the personalization stage) only contains the data files that can be accessed when processing application transactions.

Initialized application data is established when application is selected and is defined during the personalization process. Details are described in the following section.

#### **6.3.2.2 Elementary File (EF) Access Conditions**

For access to the elementary files, conditions are that command SELECT has been executed at least once and the security field has been established. Once the security field is established and the follow-up command (such as READ RECORD) or data replacement command (such as record replacement command) is transmitted to an elementary file, control of basic file access (defined by file control parameters of file control information) is compulsorily used. The details of file control parameters will be mentioned in section 6.3.4. The files using secure communication and command VERIFY (or both) as access condition can only be executed after all those conditions are met.

Access conditions of the elementary files should be applied to all commands to provide external access to IC card data, such as commands READ RECORD, GET DATA, PUT DATA and UPDATE RECORD, etc.

#### **6.3.3 File Control Information**

File control information (File Control Information, FCI) is associated with every application definition file (Application Definition File, ADF) or application elementary file (Application Elementary File, AEF) to describe features of the files.

File control information is established for every file during the personalization process. File control information of the application definition file includes file management data (File Management Data, FMD), which may include application management data, while application management data defines the security field of application.

#### **6.3.3.1 Application Management Data**

Application management data is established during the personalization process to define the initial security field, which can be stored in the file management data of the application definition file.

The security field described in application management data defines the following content:

- Resources that can be saved and accessed within the application scope, application elementary files (Application Elementary File, AEF) and internal elementary files (such as personal PIN, key and parameters)
- Commands that can be executed within the hierarchical scope of the application
- Relationship between command and resources

Security field is defined by relevant resources described by application management data. Resources excluded in the application management data should not be used by the application. For applications, security fields are independent of each other. Put it another way, definitions of security fields of different applications may be completely different.

The following two types of resources are defined:

- Data resource (described in section 6.3.3.2)
- Executable code resource (described in section 6.3.3.3)

In addition, resources can also be defined as “those not yet allocated to application”, so after personalization, the card can use corresponding commands to distribute resource to application. Resources and the relationship among resources are described by application management data (AMD).

#### **6.3.3.2 Data Resource**

Data resource can be any one of the following:

- Data file and the record
- Key
- PIN

##### **6.3.3.2.1 Data Identifier**

Data resource refers to the data element that may be included in the file. Data resources are identified by the only identifier in IC card. File is identified by the

only file identifier in IC card. The data element not included in the file is identified by the only data identifier. Any data resource required for operating application must be identified in the application management data.

For files containing data element (which can be accessed via the command defined by application management data), the relationship between SFI (which is the only identified in application and can be accessed externally) and file identifier (which is the only identified in IC card and can be accessed internally) is maintained as part of application management data.

For the data objects not included in the file (which can be accessed via command GET DATA defined by application management data), the relationship between data object tag (which can be accessed externally) and the only data identifier (which is in IC card and can be accessed internally) is maintained as part of application management data.

#### **6.3.3.2.2 Key Identifier**

Keys could be stored in the file or could also be an independent data element. Keys could not be accessed externally. For keys stored in the file, application management data maintains the file identifier required for locating the key and the reference to the key when executing command defined by application management data and encryption algorithm.

For keys not stored in the file, application management data maintains the only key identifier in IC card required for locating key when executing command defined by application management data and encryption algorithm.

#### **6.3.3.2.3 PIN/password Identifier**

PIN or password can be stored in the file or can also be an independent data element. PIN and password can only be accessed externally via command jointly defined by the application management data and secure communication.

For PIN or password stored in the file, application management data maintains the file identifier required for locating PIN/password and the quotation oriented to PIN or password when executing command defined by application management data and encryption algorithm.

For PIN/password not stored in the file, application management data maintains the only PIN/password identifier in the IC card required for locating PIN/password when executing command defined by the application management data and encryption algorithm.

#### **6.3.3.3 Executable resource**

Executable resource includes:

- Command
- Encryption algorithm

#### 6.3.3.3.1 Command Identifier

Command resources include CLA and INS bytes, which are used by the operating system to search the command location. Command resources include the property of the data the command may access and sometimes, properties of the parameters related to key and algorithm.

#### 6.3.3.3.2 Algorithm Identifier

Algorithm resources establish the relation between algorithm identifier defined for application and the actual algorithm quotation the operating system uses to locate the executable codes.

### 6.3.4 File Control Parameters

Every elementary file contains a file control parameter (FCP) in the file control information, it stores the additional information related to the access conditions of the file. The information is stored in IC card during the personalization process, and is used by IC card operating system with the application management data stored in the file control information of ADF for establishing the security field of application. The access to the elementary file is as shown in Table 26.

Table 26 Access Conditions of Elementary File

| Read       | Update | Access condition     |
|------------|--------|----------------------|
| Yes/No     | Yes/No |                      |
| Yes/No     | Yes/No | Secure communication |
| Yes/No     | Yes/No | Check                |
| (Unusable) | Yes/No | Data encryption      |

The Read column means to use the read commands, such as command READ RECORD or GET DATA, to access the internal data of basic file. “Update” column means to update the command, such as UPDATE RECORD or PUT DATA, to access the internal data of elementary files.

File control parameters point out if the data should be transmitted in encrypted or plaintext format in issuer script command UPDATE RECORD.

File control parameter is also used as a module for realizing the logic structure of application management data. Moreover, file control parameters should also describe mandatory access conditions for the elementary files of the applications on the card.

### 6.3.5 Recommended Access Conditions for Local Data on IC Card

The following recommended access conditions are applicable to the data that can be accessed by commands READ RECORD, UPDATE RECORD and GET DATA or other similar suitable commands.

- This recommendation is for the data that can only be read out with command READ RECORD. All IC card local data that have been tagged and can be externally accessed should be set in read-only state in access conditions without secure communications.
- This recommendation lists out the data that may be changed by command PUT DATA and secure communications and the data that may be read out by command GET DATA:
  - ✓ Lower limit of continuous offline transaction ('9F58')
  - ✓ Upper limit of continuous offline transaction ('9F59')
  - ✓ Continuous transaction restriction(international-national)
  - ✓ Continuous transaction restriction(international)
  - ✓ Accumulated transaction total amount restriction
  - ✓ Accumulated transaction total amount restriction(two kinds of currencies)
  - ✓ Upper limit of accumulated transaction total amount
  - ✓ Currency conversion factor
- This recommendation lists out the data that may be updated by the private command PIN CHANGE/UNBLOCK and secure communication of the application and the data that cannot be read out:
  - ✓ Reference PIN
- This recommendation lists out the data that may be read out by command GET DATA and the data that may be re-configured by command PIN CHANGE/UNBLOCK and secure communication as predetermined restriction:
  - ✓ PIN try counter

## 6.4 Key Types in card

The key types that may exist in the card:

Table 27 Key Types Stored in Card

| Key Name               | Purpose                    | Key Pattern     | Existence condition | Description                                                         |
|------------------------|----------------------------|-----------------|---------------------|---------------------------------------------------------------------|
| Application cryptogram | For generating application | Symmetrical Key | Mandatory           | It is obtained with the master key of issuer application cryptogram |



|                                       |                                                     |                  |                              |                                                                                                                                                                                                                                                                             |
|---------------------------------------|-----------------------------------------------------|------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Key                                   | cryptogram and issuer authentication in transaction |                  |                              | according to the derivation method defined in Section 9.1.4, in transaction process, process key is derived according to the method defined in Section 9.1.3 for application cryptogram generation and issuer authentication.                                               |
| Message authentication code (MAC) Key | For calculating MAC in security message             | Symmetrical Key  | Mandatory                    | It is obtained with the master key of issuer message authentication code according to the derivation method defined in Section 9.1.4, in transaction process, process key is derived according to the method defined in Section 9.1.3 for MAC calculation and verification. |
| Security message encryption Key       | For encrypting the data in script                   | Symmetrical Key  | Mandatory                    | It is obtained with the master key of issuer security message encryption according to the derivation method defined in Section 9.1.4, in transaction process, process key is derived according to the method defined in Section 9.1.3 for message encryption.               |
| Card public key/private key pair      | For offline data authentication                     | Asymmetrical Key | The card supports DDA or CDA | IC card public key certificate is generated by issuer private key pair, card public key and relevant information signature                                                                                                                                                  |

## 7 Terminal Security

### 7.1 Security Requirements For Terminal Data

#### 7.1.1 General Requirements

Two types of data usually exist in terminals:

- Common data: including time, terminal identification number and terminal transaction record, etc. Those data can be externally accessed but unauthorized modification is prohibited.
- Sensitive data: including certification authority public key, symmetrical key for PIN encryption and internal parameters of the terminal. When unauthorized, external access modification of this type of data is not allowed.

##### 7.1.1.1 Security Requirements For Common Data

Common data is usually stored in memory. When updating parameter and downloading new programs, the terminal must:

- Verify the identity of the updating party, terminal manufacturer, owners of the terminal or the third party approved owners of the terminal or agents that authorized to re-download the program.
- Check the integrity of the downloaded parameters and applied program.

Mandatory requirements on memory: No matter what the situation is, the application data of the terminal will not be changed or lost at will and it must be ensured that the data are effective.

All data related to the transaction should be stored in record form in the terminal memory. The terminal must ensure the integrity of the data.

##### 7.1.1.2 Security Requirement For Sensitive Data

Sensitive data should be stored in the terminal security module.

Security module is a hardware encryption module that provides necessary security system to prevent the data stored or processed in terminal from being illegally attacked externally.

The module is mainly responsible for storing and processing all sensitive data, which include various keys and internal parameters. Moreover, the module should also provide mandatory encrypting functions. Specific requirements on the security module hardware are not in the *UnionPay Integrated circuit card specifications-Basic Specification*.

In normal operating environment, it is required for the symmetrical key security module that the data entering or exiting the module, stored and processed in the module and data, will not be compromised or changed because of the module itself or the interface.

### **7.1.2 Physical Security Requirement of Security**

The hardware design of security module must be capable of ensuring the physical restriction of the access and grabbing of the sensitive data stored in module and the unauthorized use and modification of the security module. Once the security module is illegally attacked, the security module itself must be able to complete the deleting of internal sensitive data immediately; meanwhile, the security module must also possess sufficient security property to prevent the data from being illegally tampered. Damage or ineffectiveness of any part of the security module should not compromise the sensitive data. If the security module consists of several separated components and the data must be transmitted among those components, these components must be maintained at the same security level.

### **7.1.3 Logical Security Requirement of Security Module**

Logic design of the security module must ensure that using any single function or combined functions will not compromise the sensitive data. For some sensitive operations, there must be certain restrictions on the authority limit.

Multi pairs of certification authority public key and relevant information can be stored in the security module. Certification authority public key is usually laid in the security module before the terminal goes into service. If it is necessary to update or withdraw the certification authority public key in terminal operation process, security message must be used. Such an operation must be completed in a special authorized situation.

If information transmission is required in the security message mode, the security module must realize security message transmissions.

Security module must realize symmetrical and asymmetrical algorithm defined in Section 12 for encryption of user PIN from PINPAD to terminal and offline data authentication.

## **7.2 Security Requirements on Terminal Devices**

### **7.2.1 Anti-intrusion Devices**

An anti-intrusion device must be able to ensure that any sensitive data from the input/output devices, stored in devices or processed in devices, will not be compromised or changed by device or its interface in its normal operation environment. (Please refer to ISO 13491-2:2005 and ISO 13491-2 for further requirements on anti-intrusion devices.)

If an anti-intrusion device operates in a secure and controlled environment, the requirement for device property can be reduced as the controlled environment and device management provide device protection.

#### **7.2.1.1 Physical Security**

Anti-intrusion devices must be designed so that physical access to internal sensitive data is prohibited. Unauthorized use and modification of devices should be

restricted. Overall, targets requiring a combination of intrusion resistance, detection, and directives or reaction system, such as visible or audible alarms.

Anti-intrusion devices not in operation state must not include any encryption key or other sensitive data (such as PIN) that have been used in the previous transactions; however, it can include the authentication information for the purpose of improving anti-intrusion capability. If it is able to monitor intrusion before the devices and the key stored in the devices are re-applied, even if the devices are compromised, the security will not be affected. If the devices are designed as allowable for internal access, sensitive data must be immediately deleted when the devices are being accessed. Anti-intrusion devices depend on the detection conducted by user against attack on physical security; therefore, the devices must be designed with sufficient anti-intrusion property for a Merchant or an Acquirer to detect any intrusion.

The devices must be designed and manufactured as:

- To prevent access to the devices and to add, replace or modify anything to the software and hardware of the devices. If one has no special skills and special devices and the devices are not obviously and seriously damaged, users cannot test and modify any sensitive data and reinstall the devices.
- When truly accessing the devices, users are able to access or modify the sensitive data that are inputted and stored, or processed without being authorized.
- Common packing materials must not be used to avoid imitations and “look-alikes” from being created.
- When any component of the devices fails, no compromise of secret or sensitive data will be caused.
- If the design of the devices requires separate physical components and transmission of Cardholder instruction or data process, the protection level of all the components of the devices should be the same.
- For exchange of sensitive data such as plaintext PIN, it is mandatory to integrate different components into a single anti-intrusion shell.

#### **7.2.1.2 Logic Security**

The anti-intrusion devices must be designed so that no single function or combination of functions is able to compromise sensitive data and no multi instructions or combination of instructions is able to breach the devices, except when allowed by the security system implemented in terminal. Even if a legal function is used, there must also be sufficient logic protection so that the security of sensitive data will not be endangered. This requirement can be fulfilled via internal monitoring or control of the minimum time interval of the sensitive function used.

If the terminal can be set in a “sensitive status”, which is normally not allowed (for example, artificially installed key), such a conversion must be conducted with two

or more trusted personnel. If the PIN or other plaintext data are used to control the conversion to sensitive state, the input of PIN should also be protected in the same mode as that of other sensitive data.

To reduce the risk caused by unauthorized use of sensitive function, there must be a restriction on function usage times (as appropriate) and a restriction on time for sensitive state. Once the restrictions are reached, the devices must return to normal status.

After the transaction is over or timed-out, the anti-intrusion devices must automatically reset internal buffer memory.

### 7.2.2 PINPAD Security

A PIN pad is a tamper resistant device and supports 4-12 digit PIN entry. If the PIN pad is equipped with a display screen, it must display every input number. However, according to ISO 9564-1, the inputted PIN should not be displayed or leaked through visual or audio feedback mode.

If the terminal supports offline PIN authentication, the IC card interface device (IFD) and PINPAD should either be integrated into single anti-intrusion device or two separated anti-intrusion devices.

- If the IFD and PINPAD are integrated and the offline PIN is transmitted to card in plaintext format, the plaintext PIN is directly transmitted from the integrated PINPAD to IFD; PINPAD will not encrypt the offline PIN.
- If the IFD and PINPAD are integrated and the offline PIN is transmitted to card in plaintext format but the offline plaintext PIN is not directly transmitted from integrated PINPAD to IFD, PINPAD must encrypt the offline PIN according to ISO 9564-1 (or other equivalent mode approved by the payment system) and then transmit the encrypted offline PIN to IFD. IFD will decrypt the offline PIN and transmit it to card in plaintext.
- If the IFD and PINPAD are not integrated and the offline PIN is transmitted to card in plaintext mode, PINPAD must encrypt the offline PIN according to ISO 9564-1 (or other equivalent mode approved by the payment system) and then transmit the encrypted offline PIN to IFD. IFD will decrypt the offline PIN and transmit it to the card in plaintext.

The PIN encryption process must occur in one of the following two situations.

If the terminal supports online PIN verification, after the PIN is inputted, the PIN must be encrypted according to ISO 9564-1 in order to protect the PIN and the transmission of PIN must comply with the regulation of the payment system.

The information displayed on PINPAD to prompt that PIN should be inputted, must be generated by PINPAD. This does not limit the display of PIN information on PINPAD; however, before being displayed, other information must be approved by the PINPAD. PINPAD must decline to display any unapproved information.

For terminals with person on duty, the amount input process must be separated with the PIN input process to prevent the PIN from being accidentally displayed on the terminal display screen. Especially when the amount and PIN are inputted on the same keyboard, the amount must be two separated operations. If there is no other confirmation operation, the PIN input by the Cardholder should confirm the amount.

PINPAD must be provide privacy and confidentiality so that under normal operation, only the Cardholder can see the inputted and the displayed information. When installing and replacing PINPAD, the surrounding environment should provide sufficient privacy when the Cardholder inputs a PIN, to minimize the risk of PIN compromise to other people.

PINPAD must automatically reset the internal buffer memory if the following two situations occur:

- When the transaction has completed.
- In case of time out, including an inactive period when a PIN digit is not inputted.

### 7.3 Requirements on Terminal Key Management

#### 7.3.1 Terminal Key Type

Key types that may exist in the terminal include:

Table 28: Key Types Stored in Terminal

| Key Name                                               | Purpose                                                        | Key Format       | Existence Conditions |
|--------------------------------------------------------|----------------------------------------------------------------|------------------|----------------------|
| Certification authority public key                     | Offline data authentication                                    | Asymmetrical Key | Mandatory            |
| Maintenance Key for certification authority public key | Import, update and withdraw certification authority public key | Symmetrical Key  | Mandatory            |
| PIN encryption Key                                     | Protect the PIN of user from PINPAD to terminal                | Symmetrical Key  | Optional             |

##### 7.3.1.1 Certification Authority Public Key

It is used for offline data authentication

##### 7.3.1.2 Maintenance Key for certification authority public key

The message authentication code described in Section 9.1.2 must be used for import, update and withdraw of certification authority public key, every POS terminal should save the only maintenance key for certification authority public key, the key should be obtained with the master maintenance key for the certification authority public key of the Acquirer using the sub-key derivation method described in Section 9.1.4 for terminal counter.

The maintenance key for certification authority public key must be stored in security modulus. Please refer to Section 7.1 for security requirements.

### 7.3.2 Certification Authority Public Key Management

This section stipulates the requirements on certification authority public key in acquirer management terminal. The requirements include the following stages:

- Loading the certification authority public key into the terminal
- Storing the certification authority public key in the terminal
- Using the certification authority public key in the terminal
- Withdrawing the certification authority public key from terminal

#### 7.3.2.1 Loading the Certification Authority Public Key

If a payment system decides to import a new certification authority public key, it must ensure that the new public key will be distributed from the payment system to every Acquirer. It should ensure that the Acquirer is responsible for transmitting the new certification authority public key and relevant data to its terminal. In the process of importing the certification authority public key and the verification in security module, the transmission must be conducted via a security message system with message authentication code and the security module must return a verification code for confirmation after the verification succeeds. *UnionPay Integrated circuit card specifications-Basic Specification* will not detail the adopted security system.

The following principles apply when an Acquirer loads a new certification authority public key in its terminal.

- The terminal must verify the certification authority public key and relevant data received from the Acquirer and ensure that there is no error.
- The terminal must verify the received certification authority public key and relevant data and make sure that they are really from its legal acquirer.
- The Acquirer must confirm that the new certification authority public key has been really and correctly loaded into its terminal.

#### 7.3.2.2 Storage and Update of Certification Authority Public Key

A terminal supporting static and/or dynamic data authentication must support 6 certification authority public keys for every debit/credit application RID. All those

applications are based on the *UnionPay Integrated circuit card specifications-Basic Specification*.

Every certification authority public key should be identified with a 5-bytes RID identifying the payment system and a 1-byte certification authority public key index distributed by the payment system to some specified certification authority public key, which is unique to every RID.

The minimum set of useful data elements in the terminal is described in Table 29.

RID and certification authority public key index only identify one certification authority public key and relate with a correct payment system.

The certification authority public key algorithm identifier points out the digital signature algorithm used with corresponding certification authority public key, which is the asymmetrical algorithm that should be used in the digital signature scheme pointed out in Section 9.2.1 and Section 10.2.1 of *UnionPay Integrated circuit card specifications-Basic Specification*. Hash algorithm identifier assigns the hash algorithm that is used to generate hash result in digital signature scheme.

Certification authority public key is stored in the security modulus of terminal and can be read out at will; however, security message must be used for updating. Please refer to Section 7.3.2.1 for detailed information. Certification authority public key is verified and used to ensure that the certification authority public key and relevant data will be received accurately without any mistake. And then the terminal can use this data element to re-verify the integrity of the certification authority public key and relevant data.

The verification of the integrity of stored certification authority public key should be conducted periodically.

Table 29 The Minimum Set of Certification Authority Public Key Related Data Elements Stored in the Terminal

| Name                                              | Length | Description                                                                                     | format |
|---------------------------------------------------|--------|-------------------------------------------------------------------------------------------------|--------|
| Registered application provider identifier(RID)   | 5      | It is used to assign which payment system the certification authority public key is related to. | b      |
| Certification authority public key index          | 1      | It is used to assign the certification authority public key with RID.                           | b      |
| Certification authority Hash algorithm identifier | 1      | It is used to identify the Hash algorithm generating hash result in digital signature scheme.   | b      |
| Certification authority                           | 1      | It is used to identify the digital                                                              | b      |



|                                                |                                |                                                                                                                                                                                                                                                                                                                   |   |
|------------------------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| public key algorithm identifier                |                                | signature algorithm used on certification authority public key.                                                                                                                                                                                                                                                   |   |
| Certification authority public key modulus     | Variable<br>The maximum is 248 | Value of the certification authority public key modulus part.                                                                                                                                                                                                                                                     | b |
| Certification authority public key index       | 1 or 3                         | Value of the certification authority public key index part, equal to 3 or $2^{16}+1$ .                                                                                                                                                                                                                            | b |
| Certification authority public key check value | 20                             | Verification value obtained from the connecting calculation of all parts of certification authority public key (RID, certification authority public key index, certification authority public key modulus and certification authority public key index number) via the hash algorithm designated in Section 10.3. | b |

#### 7.3.2.3 Usage of Certification Authority Public Key

The use of certification authority public key in transaction must observe *UnionPay Integrated circuit card specifications-Basic Specification*.

#### 7.3.2.4 Revocation of Certification Authority Public Key

When a payment system decides to withdraw one of its certification authority public key, the Acquirer must ensure that after a determined period of time its terminal will no longer use that certification authority public key in transaction for static and dynamic data authentication.

The following principles apply when the Acquirer withdraws the certification authority public key from its terminal:

- The terminal must verify that there is no mistake in the revocation notice received from the Acquirer.
- The terminal must verify that the received revocation notice is really from its legal acquirer.
- The Acquirer must confirm that a specified certification authority public key has been correctly withdrawn from its terminal.

The revocation instruction of certification authority public key should also be transmitted via security message. Please refer to Section 10 for more information about the certification authority public key revocation schedule.

## 8 Key Management System

Both the payment system and Issuer need to establish a complete set of key management system, and the payment system needs to establish a certification authority responsible for managing the certification authority public key and private key pair used for offline data authentication. Issuer needs to establish a complete set of key management system used for offline data authentication and transaction procedure.

### 8.1 Certification Authority Public Key Management

This section defines the overall framework of principle and tactics for the management of certification authority public key in payment system and the certification authority public key is used in the static and dynamic data authentication indicated in *UnionPay Integrated circuit card specifications-Basic Specifications*.

#### 8.1.1 Life Cycle of Certification Authority Public Key

The life cycle of certification authority public key in common environment can be divided into the following continuous phases:

- Planning
- Generation
- Distribution
- Usage
- Revocation (as planned)

##### 8.1.1.1 Planning

In the planning phase, the payment system investigates and studies the requirements for importing new certification authority public key pair. The requirements include the quantity of required keys and the parameters of the keys.

One important part of the planning phase is to determine the expected life cycle of existing key, or the new key that will be adopted via assessing the security of asymmetrical encryption algorithm. Such assessment guides the configuration of the length and expiry date of the new key, the potential modification of the expiry date of existing key, and the overall planned key replacement.

##### 8.1.1.2 Generation

If the result of the planning stage is to import new certification authority public key pair, the keys must be generated by the payment system. More accurately speaking, the certification authority of the payment system (payment system operated by the organization is highly secure, both physically and logically) will generate the required certification authority private key/public key pair in a secure mode for use in the future.

After the keys are generated, the private security of the certification authority private key and the integrity of the certification authority public key and private key must also be ensured.

#### 8.1.1.3 Distribution

In the key distribution stage, the certification authority of the payment system should distribute the newly generated certification authority public key to its member Issuers and Acquirers for the following uses:

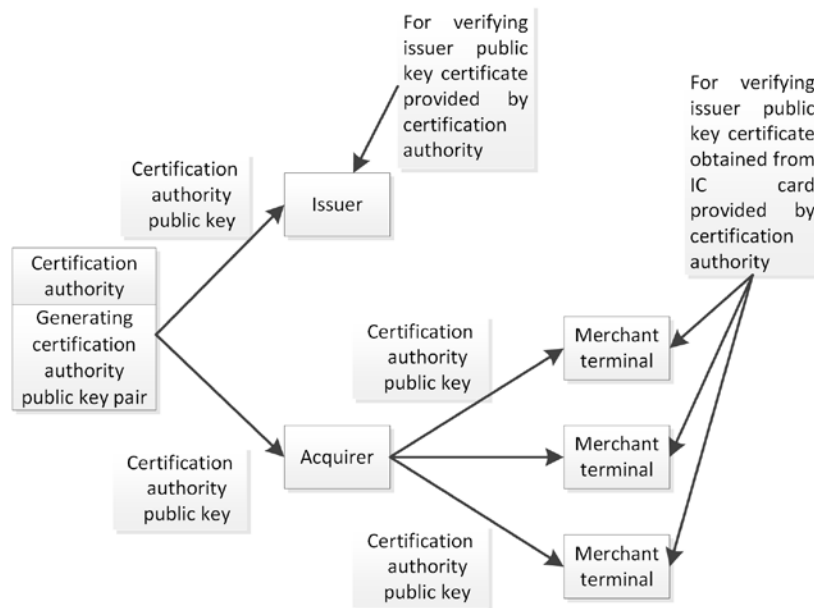


Figure 7 Distribution of Certification Authority Public Key

- For the Issuer, it verifies the issuer public key certificate provided by the certification authority in use stage.
- For the Acquirer, it securely imports certification authority public key in the Merchant terminal.

To avoid importing fake certification authority public key, the interfaces between the certification authorities of the payment system, Issuer and Acquirer must ensure the integrity of the certification authority public key distribution.

#### 8.1.1.4 Usage

The certification authority public key is used by merchant terminal for completing static and dynamic data authentication. The certification authority private key is used by the certification authority of payment system for generating issuer public key certificate. More specifically, the following interactive operations occur:

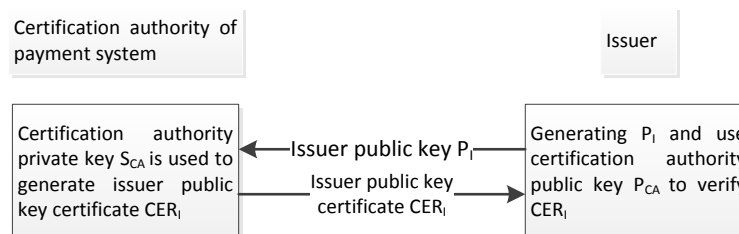


Figure 8 Distribution of Issuer Public Key

- The Issuer generates its own issuer public key and transmits it to the certification authority of the payment system.
- The certification authority of the payment system uses certification authority private key to sign the issuer public key in order to generate issuer public key certificate and transmits it back to the Issuer.
- The Issuer uses certification authority public key to verify whether the issuer public key certificate is correct or not. If it is correct, the Issuer can use it as part of a personalized IC card data.

To avoid the import of fake certification authority public key and the interfaces between the certification authority of the payment system, the Issuer and the Acquirer must ensure the integrity of the submitted issuer public key.

#### 8.1.1.5 Revocation (As Planned)

Once the certification authority public keys reach the expiry date set at the planning stage, it must be deleted from the service. In practice, it means:

- After the expiry date, the issuer public key certificate generated from the certification authority private key is no longer effective; therefore, the Issuer must ensure that the use of the personalized IC card with such an issuer public key certificate should be terminated before the expiry date of the certification authority public key pair.
- Before the expiry date, the certification authority of the payment system should stop using the corresponding certification authority private key to sign the issuer public key.

After the expiry date, the Acquirer should delete the certification authority public key from the terminal within the stipulated time limit.

#### 8.1.2 Compromise of Certification Authority Public Key

When the certification authority public key pair is compromised, emergency state must be implemented, which may finally result in revocation of certification authority public key in advance before the planned expiry date. In such a case, there will be some additional phases in the life cycle of key. They are described below:

- Detection
- Assessment

- Decision
- Revocation (in advance)

#### **8.1.2.1 Detection**

Certification authority public key pair can be a true compromise. For example, confirmed security of the certification authority of the payment system can be damaged, or the analyzed confirmed key exposed via cryptanalysis. In addition, the compromise can also be:

- Suspected: System monitoring and member or Cardholder complaints indicate that fraud transactions might have occurred, and that may be caused by the compromise of key; however, that has not been confirmed.
- Potential: Cryptanalysis technique, such as factorization, has developed to the level that any key of the current length can be analyzed and explained with existing resources; however, there is no evidence indicating that it has already occurred.

Detection of key compromise includes the awareness of illegal physical intrusion in the payment system, the offline fraud transaction report provided by the fraud and risk management system installed by the payment system and its member and the information concerning factorization technique development collected by the key organization.

#### **8.1.2.2 Assessment**

The assessment of a certification authority public key compromise (or potential) includes the technique, risk, fraud and the most important commercial impact on the payment system and the members. The result of assessment includes the possible series of actions decided after the confirmation of compromise, overall cost and the risk brought by the compromise are determined as well as the assessment result provided for supporting decision.

#### **8.1.2.3 Decision**

According to the assessment result, the payment system will decide to take a series of actions against the key compromise. In the worst situation, the decision will include the revocation of certification authority public key in advance before the planned expiry date.

#### **8.1.2.4 Revocation (In advance)**

After deciding to revoke the certification authority public key, members of the payment system should be notified with the new expiry date of the corresponding key. Afterwards, the processing will be the same as the revocation (as planned) described in Section 8.1.1.5.

### **8.1.3 Strategies of Certification Authority Key Management**

#### **8.1.3.1 Planning Phase**

The main tasks in this phase include:

1. Analysis of the attack-resistant public key and the demand for a new public key pair.

2. Deciding the quantity and parameters of the newly demanded public key pair.

The parameters include:

- Selection of the public key length
- Expiry date of the public key

The following strategies should be adopted for the public key expiry date:

- December 31st should be the planned expiry date for all certification authority public keys.
- The Acquirer should have a transitional period of six months after the planned expiry date (until June 30th of next calendar year) to revoke overdue keys from all terminals. Mandatory key revocation should not appear before the end of the transitional period. The revocation can be postponed based on the judgment of the payment system.
- All new certification authority public keys should be issued before December 31st.
- The Acquirer should have a transitional period of six months (until June 30th of the next calendar year) to install the new key on all terminals.
- The Acquirer should have a period of six month (until June 30th of the next calendar year) to install the new key on all terminals. New certification authority public keys should be issued any time before December 31st to leave enough time for the public key installation.
- The payment system will not make the new certification authority public key effective for legal transactions before January 1<sup>st</sup> of the next year.
- In case of an advance revocation, the transitional period of six months for revoking the key from all terminals remains unchanged; however, the fixed December 31<sup>st</sup> date will no longer apply. The payment system is responsible for notifying the members of the key revocation and time arrangement.

#### **8.1.3.2 Generation Phase**

The main tasks in this phase include:

1. The generation of a certification authority private key/public key pair in a secure mode by the certification authority.

2. For every registered application provider identifier (RID), the certification authority public key index directing to a specified certification authority public key pair possesses one value only. The certification authority index value of a specified key cannot be changed.

#### **8.1.3.3 Distribution Phase**

The main tasks in this phase include:

1. The distribution of the newly generated certification authority public key to its member Issuers and Acquirers by the certification authority.
2. The import of the public keys into merchant terminals by the Acquirers.

#### **8.1.3.4 Usage Phase**

The main tasks in this phase include:

1. Sign and issue issuer certificate for Issuers.
2. The Issuer uses the certification authority public key to verify whether the received issuer public key certificate is correct or not and load it to IC card via card personalization. The following tactics should be adopted for signing with the certification authority private key:
  - After the key is issued to Acquirers for at least six months, the certification authority begins to use the secret key in the certification authority public key pair for signing.
  - The expiry date of the user's IC card issued by the Issuer must be no later than the expiry date of the issuer public key certificate on the card, and must also be no later than the issued (when the card is issued) revocation date of the certification authority public key used to generate the issuer public key certificate.
  - The expiry date of an issuer public key certificate must be no later than the issued (when the certificate is issued) revocation date of the certification authority public key used to generate the issuer public key certificate.
  - The expiry date of an IC card public key certificate must be no later than the revocation date of the issuer public key used in generating the IC card public key certificate.

#### **8.1.3.5 Detection Phase**

The main tasks in this phase include:

1. Monitoring the security of the certification authority private key. Private key compromise forms include physical, logical, suspected, potential, and confirmed compromise forms.

#### **8.1.3.6 Assessment Phase**

This phase is only applicable to advanced revocation.

The main task in this phase is to assess the impact caused by the public key compromise on commercial business, including:

- Confirmation of compromise

- Determination of possible series actions to be taken
- Comparison of the cost of actions and the cost and risk brought about by the compromise
- Submitting assessment result to support decision

#### **8.1.3.7 Decision Phase**

This phase is only applicable to advanced revocation. As a result of the assessment phase, the payment system decides the series of actions taken against the compromise of certification authority public key pair in this phase.

#### **8.1.3.8 Revocation Phase**

The main tasks in this phase include:

1. A key management process of revoking a key from service and handling the left-over issues after its use. Key revocation can be conducted as planned or in advance. Based on the situation of certification authority public key pair, the revocation means that the private key will no longer be used to generate issuer public key certificate.
2. Copies of the public key are deleted from merchant terminals.

Revocation of public key should adopt the following tactics:

- December 31<sup>st</sup> should be set as the expiry date for all certification authority public keys. The Acquirer should have a transitional period of six months (until June 30<sup>th</sup> of the next calendar year) to revoke abrogated key. Mandatory key revocation should not appear before the transitional period is terminated and the revocation can be postponed based on the judgment of the payment system.
- Observing the regulations of payment system, revocation of certification authority public key pair means that the public key part should be revoked from the service of all terminals within six months.

In an advanced revocation, the reserved time for import and revocation are the same as that for planned revocation; however, the date of revocation is determined based on the judgment of the payment system.

### **8.2 Management of Issuer Public Key**

The public key management strategies of issuer public key management can be worked out in reference to the certification authority public key management strategies.

Before applying the certification authority for public key certificate, the Issuer should decide on the key management which include:

- Required number of issuer public keys to generate
- Length of the generated key. The Issuer Public Key Length must not be greater than that of the longest key of certification authority.



- Expiry date of every key. The expiry date of key must be no later than that of the public key the certification authority uses to sign and issue certificate.
- Key index

After the Issuers have worked out their key management tactics and a pair of public keys have been generated, the issuer public key should be submitted to the certification authority. When the certification authority receives multi-issuer public keys, appropriate certificate should be selected to load into the card.

### 8.3 Management of Issuer Symmetrical Key

#### 8.3.1 Requirement of Security

The key management system must have functions such as user security management, devices security management, key security management, and audit management:

- User security management exercises the control and management of the authority limit of system operators to prevent the system from being illegally used beyond one's authority.
- Devices security management exercises the security management of encryptor in system such as encryption devices. Encryptor must be configured with the corresponding capability to prevent the hardware from being attacked and ensure that the key stored in encryptor will not be illegally read out or obtained.
- Rational security design should be adopted for key security management to ensure the security of key in phases such as storage, transmission and use, etc.
- Audit management is used for the security audit and management of system operation log and other relevant information.

#### 8.3.2 Functional Requirements

##### 8.3.2.1 Key Types

Symmetrical Key Types of system management are shown in the table below:

Table 30 Symmetrical Key Types of Management

| Key type                              | Purpose                                                                                                     | Length   |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------|----------|
| Master Key of application cryptogram  | Generating IC card application cryptogram sub-key for generation and verification of application cryptogram | 16-bytes |
| Message authentication code (MAC) key | Generating IC card MAC sub-key for generation and verification of message authentication code               | 16-bytes |

|                                    |                                                                                      |          |
|------------------------------------|--------------------------------------------------------------------------------------|----------|
| Key of security message encryption | Generating IC card encryption sub-key for encrypting and decrypting security message | 16-bytes |
|------------------------------------|--------------------------------------------------------------------------------------|----------|

IC card sub-key can be decentralized from issuer master key, and corresponding process key is derived from sub-key in the transaction process, where MAC key is used to generate security verification code (MAC) of message for security message commands, such as secure updating of data and issuer script, etc. Encryption key is used to encrypt security message and AC key is used to make encryption calculation of TC, ARQC, AAC, and ARPC.

### 8.3.2.2 Key Management

The system must realize the following key management functions:

- Function of key generation: A key is generated according to the user's input through a specified key input algorithm. The generation of key can adopt seed encoding mode or random generation mode.
- Key transmission function: It is the function of transmitting system key securely to transaction authentication devices or encryption devices for card issuance.
- Key back-up and recovery function: It provides the back-up and recovery function for system key so that when the system collapses catastrophic recovery of the system key can be achieved.
- Updating and revocation of key.

### 8.3.2.3 Function of Encryption Devices

The system devices must be able to realize the following functions:

- Key derivation function: The only IC card sub-key is decentralized from the issuer master key stored in encryption devices according to the derivation method defined in Section 9.1.4.
- Process key generation function: Process key is decentralized based on sub-key and input data according to the derivation method defined in Section 9.1.3.
- Data encryption function: Data encryption and decryption are performed according to sub-key and process key.
- MAC generation function: Data verification code is generated according to the MAC process key and the data to calculate.
- ARQC verification function: The correctness of ARQC is verified according to the transaction data.
- ARPC generation function: ARPC is generated according to the transaction data.

## 9 Security Systems

### 9.1 Symmetric Encryption System

#### 9.1.1 Encryption and Decryption

Data encryption adopts block encryption algorithm with a block length of 64-bits (8-bytes) or 128-bits (16-bytes) and can be in electronic cipher book (ECB) or cipher block connection (CBC) mode. ECB mode is selected as the encryption and decryption mode in the *UnionPay Integrated circuit card specifications-Basic Specifications*.

The steps to encrypt message MSG of any length with encryption process key  $K_S$  are as follows:

##### 1. Padding and blocking

- If the length of message MSG is not the round multiple of block length, add a '80' byte at the right end of MSG and then add minimum '00' byte at the right end to make the length of result message  $MSG := (MSG \parallel '80' \parallel '00' \parallel '00' \parallel \dots \parallel '00')$  a round multiple of the block length.
- If the length of message MSG is the round multiple of block length, no padding is performed for the data.

The data to be encrypted should firstly be formatted into the data block of the following forms:

- Length of plaintext data, excluding pad characters.
- Plaintext data
- Pad characters(according to the padding mode described above)

And then MSG is divided into 8-bytes or 16-bytes blocks,  $X_1, X_2, \dots, X_K$ .

##### 2. Cryptogram calculation

###### ECB mode

Blocks  $X_1, X_2, \dots, X_K$  are encrypted into blocks  $Y_1, Y_2, \dots, Y_K$  of the block length with encryption process Key  $K_S$  via blocking encryption algorithm of ECB mode.

Therefore, when  $i = 1, 2, \dots, K$ , calculate separately:

$$Y_i := \text{ALG}(K_S)[X_i].$$

###### CBC mode

Blocks  $X_1, X_2, \dots, X_K$  are encrypted into blocks  $Y_1, Y_2, \dots, Y_K$  of the block length with encryption process Key  $K_S$  via blocking encryption algorithm of CBC mode.

Therefore, when  $i = 1, 2, \dots, K$ , calculate separately:

$$Y_i := \text{ALG}(K_S)[X_i \oplus Y_{i-1}],$$

The original value of  $Y_0$  is:

- Corresponding to 64-bits block encryption algorithm  $Y_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$
- Corresponding to 128-bits block encryption algorithm  $Y_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$

Recorded as:

$$Y := (Y_1 \parallel Y_2 \parallel \dots \parallel Y_K) = \text{ENC}(K_S)[\text{MSG}].$$

The decryption process is as follows:

#### 1. Cryptogram decryption

##### ECB mode

When  $i = 1, 2, \dots, K$ , calculate separately:

$$X_i := \text{ALG}^{-1}(K_S)[Y_i]$$

##### CBC mode

When  $i = 1, 2, \dots, K$ , calculate separately:

$$X_i := \text{ALG}^{-1}(K_S)[Y_i] \oplus Y_{i-1},$$

The original value of  $Y_0$  is:

- Corresponding to 64-bits block encryption algorithm  $Y_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$
- Corresponding to 128-bits block encryption algorithm  $Y_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$

2. To obtain the original message MSG, connect blocks  $X_1, X_2, \dots, X_K$  and if padding is adopted (please see above context), delete the ending of byte string ('80'  $\parallel$  '00'  $\parallel$  '00'  $\parallel$  ...  $\parallel$  '00') in final block  $X_K$

Recorded as:

$$\text{MSG} = \text{DEC}(K_S)[Y].$$

### 9.1.2 Message Authentication Code

#### 9.1.2.1 MAC calculation method based on 64-bits block encryption algorithm

The MAC calculation method can be seen in Appendix A.1 and the length of MAC is 4-bytes.

64-bits block encryption algorithm of CBC mode is adopted to calculate MAC of s-bytes ( $4 \leq s \leq 8$ ) according to specification ISO/IEC 9797-1. To put it more precisely, steps of calculating the MAC value S for message MSG with any length using MAC process key  $K_S$  are as follows:

#### 1. Padding and blocking

Message MSG is padded according to ISO/IEC 7816-4:2005 (equivalent to mode 2 in ISO/IEC 9797-1); therefore, one '80' byte is added mandatorily at the right end of MSG and then add minimum '00' byte at the right end to make the length of result message  $MSG := (MSG \parallel '80' \parallel '00' \parallel '00' \parallel \dots \parallel '00')$  a round multiple of 8-bytes.

And then MSG is unpacked into 8-bytes blocks,  $X_1, X_2, \dots, X_K$ .

#### 2. MAC process key

MAC process key  $K_S$  can both only include the leftmost key block  $K_S = K_{SL}$  and can also be connected between the leftmost key block and the rightmost key block into  $K_S = (K_{SL} \parallel K_{SR})$ .

#### 3. Cryptogram calculation

8-bytes  $X_1, X_2, \dots, X_K$  are processed with the leftmost block of MAC process key  $K_{SL}$  in block encryption of CBC mode:

$H_i := \text{ALG}(K_{SL})[X_i \oplus H_{i-1}]$ , here  $i = 1, 2, \dots, K$ .

The original value of  $H_0$  is  $H_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$ .

Use one of the following two methods to calculate 8-bytes block  $H_{K+1}$ .

1) According to ISO/IEC 9797-1 algorithm 1:  $H_{K+1} := H_K$ .

2) According to ISO/IEC 9797-1 algorithm 3:  $H_{K+1} := \text{ALG}(K_{SL})[\text{ALG}^{-1}(K_{SR})[H_K]]$ .

The second calculation method is adopted in *UnionPay Integrated circuit card specifications-Basic Specification*.

The MAC value S is equal to s top bytes of  $H_{K+1}$ .

#### 9.1.2.2 MAC Calculation Method Based on 128-bits Block Encryption Algorithm

The steps used in calculating a value S of s-bytes MAC ( $4 \leq s \leq 8$ ) for message MSG of any length via using 128-bits block encryption algorithm of CBC mode and MAC process key  $K_S$  are as follows:

#### 1. Padding and blocking

Message MSG is padded according to ISO/IEC 7816-4:2005 (equivalent to mode 2 in ISO/IEC 9797-1); therefore, one '80' byte is added mandatorily at the

right end of MSG and then add minimum '00' byte at the right end to make the length of result message  $MSG := (MSG \parallel '80' \parallel 00 \parallel '00' \parallel \dots \parallel '00')$  a round multiple of 16-bytes.

And then MSG is unpacked into 16-bytes blocks,  $X_1, X_2, \dots, X_K$ .

## 2. MAC process key

The length of MAC process key  $K_S$  is 16-bytes.

## 3. Cryptogram calculation

16-bytes  $X_1, X_2, \dots, X_K$  are processed with MAC process key in block encryption of CBC mode:

$H_i := \text{ALG}(K)[X_i \oplus H_{i-1}]$ , here  $i = 1, 2, \dots, K$ .

The original value of  $H_0$  is  $H_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$ .

Use following method to calculate 8-bytes block  $H_{K+1}$ .

$H_{K+1} := H_{KL} \oplus H_{KR}$ .

MAC value S is equal to top bytes of  $H_{K+1}$ .

### 9.1.3 Session Key Derivation

#### 9.1.3.1 Session key derivation method based on 64-bits block encryption algorithm

The generation of MAC and data encryption session key is as follows: (in this section, they are jointly called "session Key A" and "session Key B").

##### 1. Single length DES session key

Please refer to the method defined in Appendix B3 of "Application Specification (V1.0) of China Financial Integrated Circuit (IC) Card Specification" for the derivation method of single length DES session key.

The first step: Card/Issuer decides to use MAC keys A and B or data encryption keys A and B for conducting selected algorithm processing (afterwards jointly called "Key A" and "Key B").

The second step: Use a hexadecimal number '0' to pad at the left side of the current ATC to 8-bytes and use key A and key B to conduct 3-DES calculation of the data as shown in Fig. 9 in order to generate session key A.

$Z := 3\text{-DES}(\text{Key})['00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel \text{ATC}]$

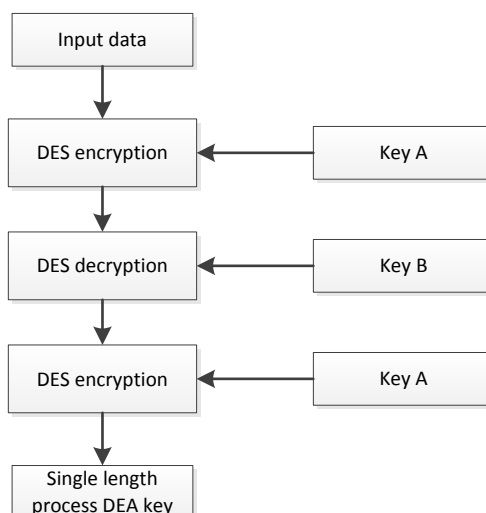


Figure 9 Generation of single length session key

## 2. Double-length DES Session key

The first step: Card/Issuer decides to use MAC keys A and B or data encryption keys A and B for performing the selected algorithm (afterwards jointly called “Key A” and “Key B”).

The second step: Use a hexadecimal number ‘0’ to pad at the left side of current ATC to 8-bytes and use key A and key B to conduct 3-DES calculation of the data as shown in Fig. 9 to generate session key A.

Use a hexadecimal number ‘0’ to pad to 8-bytes at the left side of current ATC which has conducted XOR with hexadecimal value FFFF, and then use the same method to conduct 3-DES calculation as shown in Fig. 9 to generate session key B.

$$ZL := 3\text{-DES}(\text{Key})[‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel \text{ATC} ]$$

$$ZR := 3\text{-DES}(\text{Key})[‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel ‘00’ \parallel (\text{ATC} \oplus \text{‘FFFF’}) ]$$

To meet the requirements of DES key odd verification, the minimum bit of every byte of DES key should be designed so that it is able to ensure that every one of the 8 or 16 bytes of the key has odd non-zero digit.

### 9.1.3.2 Session Key Derivation Method based on 128-bits Block Encryption Algorithm

The generation of MAC and data encryption session key is as follows:

The first step: Card/Issuer decides to use MAC key or data encryption key for performing selected algorithm processing.

The second step: Use a hexadecimal number ‘0’ to pad 8-bytes at the left side of the current ATC. The result is recorded as data source A, use a hexadecimal number ‘0’ to pad 8-bytes at the left side of current ATC which has conducted XOR with hexadecimal value FFFF and the result is recorded as data source B. Connect data

source A and data source B in concatenation and then generate session key by making the data calculating as shown in Fig. 10 with the selected key.

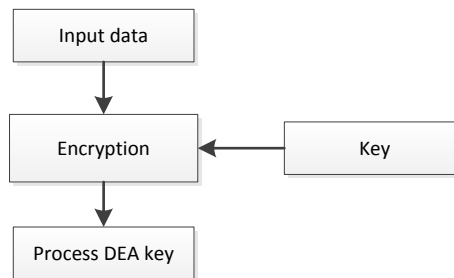
$$Z := \text{ALG}(\text{Key})[ ['00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel \text{ATC} \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel (\text{ATC} \oplus \text{'FFFF'}) ] ]$$


Figure 10 Generation of 128-bits Block Encryption Algorithm Session Key

#### 9.1.4 Derivation of sub-key

This section assigns a method of generating an IC card sub-key via derivation with a 16-bytes issuer master key IMK for cryptogram generation, issuer authentication, and security message.

The method takes as input the rightmost 16 digits of the concatenation PAN with PAN Sequence Number (if the PAN Sequence Number is not present, then it is replaced by a '00' byte), plus a 16-byte Issuer Master Key IMK, and produce the 16-byte IC Card sub-key MK in the following way.

1. If the length of primary account number and the PAN Sequence Number X is less than 16 digits, pad it to the left with hexadecimal zeros in order to obtain an 8-bytes numeric format Y. If the length of X is at least 16 digits, then Y consists of the rightmost 16 digits of X.

2. Calculate 2 8-byte numbers.

a) Calculation based on 64-bits block encryption algorithm

$$Z_L := \text{ALG}(\text{IMK})[Y]$$

And

$$Z_R := \text{ALG}(\text{IMK})[Y \oplus (\text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'})]$$

And define

$$Z := (Z_L \parallel Z_R)$$

b) Calculation based on 128-bits block encryption algorithm

$$Z := \text{ALG}(\text{IMK})[Y \parallel (Y \oplus (\text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'} \parallel \text{'FF'}))]$$

The IC card sub-key MK of 16-bytes is equal to Z, additionally, for DES algorithm, the minimum bit of every byte of Z should be configured as such that is able to



ensure that every byte of the 16 bytes of MK has an odd non-zero digit (in order to meet the requirements of DES key odd verification).

## 9.2 Asymmetrical Encryption System

### 9.2.1 Digital Signature Scheme for Message Recovery

This section describes the scheme to recover digital signature scheme by using the given message of hash function according to specification ISO/IEC 9796-2, which is used by static and dynamic data authentication in *UnionPay Integrated circuit card specifications-Basic Specifications*.

#### 9.2.1.1 Algorithm

The digital signature scheme uses the following algorithms:

- A reversible asymmetrical algorithm consisting of one signature function  $\text{Sign}(S_K)[\ ]$  depending on private key  $S_K$  and one recovery function  $\text{Recover}(P_K)[\ ]$  depending on public key  $P_K$ . The two functions both map  $N$  bytes into  $N$  bytes, and any number  $X$  with  $N$  bytes has characteristics as follows.

$$\text{Recover}(P_K)[\text{Sign}(S_K)[X]] = X$$

- A hash algorithm maps the message of any length into a hash value of 20 bytes.

#### 9.2.1.2 Generation of Digital Signature

The process of calculating signature  $S$  for message  $\text{MSG}$  consisting of data of any length  $L$  with at least  $N-21$  bytes length is as follows:

1. Calculate the hash value of 20-bytes message  $M$ ,  $H := \text{Hash}[\text{MSG}]$ .
2. Unpack  $\text{MSG}$  into two parts  $\text{MSG} = (\text{MSG1} \parallel \text{MSG2})$ , where  $\text{MSG1}$  consists of the leftmost (top bit)  $N-22$  bytes of  $\text{MSG}$  and  $\text{MSG2}$  consists of the rest (minimum bit)  $L - N + 22$  bytes of  $\text{MSG}$ .
3. Define a byte  $B := '6A'$ .
4. Define a byte  $E := 'BC'$ .
5. Define block  $X$  of  $N$ -byte as connection of block  $B$ ,  $\text{MSG1}$ ,  $H$  and  $E$ , therefore,  

$$X := (B \parallel \text{MSG1} \parallel H \parallel E)$$
6. Digital signature  $S$  is defined as a number of  $N$ -bytes.

$$S := \text{Sign}(S_K)[X]$$

#### 9.2.1.3 Digital Signature Verification

The corresponding signature verification process is as follows:

1. Examine if the digital signature consists of  $N$ -bytes.
2. Number  $X$  with  $N$ -bytes is recovered from digital signature  $S$ .

$$X = \text{Recover}(P_K)[S]$$

3. Unpack block X into  $X = (B \parallel \text{MSG1} \parallel H \parallel E)$ , where:

B is one byte long.

H is 20-bytes long.

E is one byte long.

MSG1 consists of the remaining N-22 bytes.

4. Examine if byte B is equal to '6A'.

5. Examine if byte E is equal to 'BC'.

6. Calculate  $\text{MSG} = (\text{MSG1} \parallel \text{MSG2})$  and examine if it complies with  $H = \text{Hash}[\text{MSG}]$ .

The received message is authentic only when all the examinations are correct.

## 10 Approved Algorithm

### 10.1 Symmetric Encryption Algorithm

#### 10.1.1 DES

DES algorithm is a calculation conducted with 64-bits block as the unit and the length of key is 8-bytes. The algorithm is allowed to be used for security message transmission and MAC system cryptogram calculation. The detailed process of the algorithm is defined in ISO 8731-1, ISO 8732 and ISO/IEC 10116.

3-DES encryption refers to encrypting 8-bytes plaintext data block with double-length (16-bytes) Key  $K=(KL\|KR)$  into cryptogram data block, as shown below:

$$Y = \text{DES}(KL)[\text{DES-1}(KR)[\text{DES}(KL)[X]]]$$

The decrypting mode is as follows:

$$X = \text{DES-1}(KL)[\text{DES}(KR)[\text{DES-1}(KL)[Y]]]$$

Single DES is only allowed to be used in MAC system of algorithm 3 in ISO 9797-1 shown in Section 9.1.2.1 (3 DES is used in the last block).

#### 10.1.2 SSF33

SSF33 algorithm is a calculation conducted with 128-bits block as the unit and the length of key is 16-bytes. The algorithm can also be used for security message transmission and MAC system cryptogram calculation.

Table 31 Comparison of SSF33 and DES

|                 | DES     | 3-DES    | SSF33    |
|-----------------|---------|----------|----------|
| Length of Key   | 8-bytes | 16-bytes | 16-bytes |
| Length of block | 8-bytes | 8-bytes  | 16-bytes |

SSF33 algorithm and symmetrical encryption system based on 3-DES use the keys of the same length, which are compatible with the original key management based on 3-DES, the difference exists in the different lengths of blocks, and during encrypting and calculating MAC and key derivation the padding and calculating modes are different, however, the lengths of message authentication code and key derivation output result should be identical with 3-DES algorithm.

### 10.2 Asymmetrical Encryption Algorithm

#### 10.2.1 RSA

The reversible algorithm is an approved algorithm used for encrypting and generating digital signature. The value of public key index can only be 3 and  $2^{16} + 1$ .

The lengths of cryptogram and digital signature generated from the algorithm should be equal to that of modulus.

Table 32 Mandatory Upper Limit of Modulus Length Bytes

| Description                                | Maximum length |
|--------------------------------------------|----------------|
| Certification authority public key modulus | 248-bytes      |
| Issuer public key modulus                  | 248-bytes      |
| IC card public key modulus                 | 248-bytes      |

The length of certification authority public key modulus  $N_{CA}$ , the length of issuer public key modulus  $N_I$  and the length of IC card public key modulus  $N_{IC}$  must comply with  $N_{IC} \leq N_I \leq N_{CA}$  and  $N_{PE} \leq N_I \leq N_{CA}$ .

Note: It is stipulated in chapter III, volume III of EMV specification that the maximum length of a record in IC card must not exceed 254-bytes (including Tag and Length); therefore, the actual length of IC card public key and issuer public key should be less than the maximum length 248-bytes. It is stipulated in volume I of EMV specification that the maximum length of command data is 255-bytes and that of response data is 256-bytes. Using dynamic signature data as IC card response data also restricts the maximum length of IC card public key.

If the card supports DDA and CDA, including the length of the record template of IC card certificate, which means that the length of IC card public key certificate plus the Tag and length of certificate (tag '9F46') and record template (tag '70'), the total length should not exceed 254-bytes, so the length of IC card public key should not exceed 247-bytes; therefore, the maximum length of issuer public key should not exceed 247-bytes either.

The maximum length of IC card public key should be between 205 and 240 according to different length of issuer application data. If command Generate AC contains other optional data, the maximum length of IC card public key should be subtracted by the lengths of those data (including Tag and Length). If the card application supports INTERNAL AUTHENTICATION format II, the maximum length of IC card public key should also be additionally subtracted by 7-bytes.

When selecting the length of public key modulus, comparison of the life cycle of key and the expected factorization process should be considered.

The values of issuer public key exponent and IC card public key exponent should be determined by the Issuer. The public key indexes of certification authority, Issuer and IC card must equal to 3 or  $2^{16} + 1$ .

The public key algorithm identifier for identifying this digital signature algorithm must be the hexadecimal code of '01'.

The key, signature, and recovery function of RSA algorithm using odd public key index are described in the following paragraphs.

#### 10.2.1.1 Key

The secret key  $S_K$  of RSA digital signature scheme using odd public key index  $e$  consists of two prime numbers  $p$  and  $q$ , complying with:

$p-1$ ,  $q-1$  and  $e$  are prime to each other,

and private key index  $d$ , complying with:

$$ed \equiv 1 \pmod{(p-1)(q-1)}$$

Composition:

Corresponding public key  $P_K$  consists of public key modulus  $n = pq$  and public key index  $e$ .

#### 10.2.1.2 Signature Function

RSA signature function using odd public key index is defined as:

$$S = \text{Sign}(S_K)[X] := X^d \pmod{n}, 0 < X < n$$

Here:  $X$  is the data used for signing and  $S$  is the corresponding digital signature.

#### 10.2.1.3 Recover Function

RSA recover function using odd public key index is defined as:

$$X = \text{Recover}(P_K)[S] := S^e \pmod{n}.$$

#### 10.2.1.4 Generation of Key

The payment system and the Issuer must be responsible for the security of the respective RSA public/secret key generation process.

### 10.3 Hash Algorithm

#### 10.3.1 SHA-1

For SHA-1 algorithm see *ISO/IEC 10118-3*. SHA-1 generates a Hash value of 20-bytes for the input of message of any length.

The marking code of this hash algorithm is a hexadecimal '01'.

## Appendix A (Informative Appendix) MAC Calculation

### A.1 Calculation of MAC/TA

Generation of MAC/TAC uses the following single DEA method:

Step 1: Set an 8 byte initial vector as hexadecimal “0x 00 00 00 00 00 00 00 00”.

Step 2: Concatenate all data entered in specified order as a single data block.

Step 3: Segment the concatenated data block into 8-byte data blocks, labeled as D1, D2, D3, and D4, etc. At the end, the remaining bytes form a last data block that is less than or equal to 8 bytes in length.

Step 4: If the last data block is 8 bytes long, then append an 8-byte long data block, with hexadecimal value of “0x 80 00 00 00 00 00 00 00”. If the last block has fewer than 8-bytes, then append a byte with hexadecimal value of ‘0x80’. If the data block is 8 bytes long after the appending, then skip to step 5. If the data block is still less than 8 bytes long after the appending, then keep appending hexadecimal value of ‘0x80’ until the data block is 8 bytes long.

Step 5: MAC generation uses the data block created through the method above, processed with the session key. For session key generation, see Figure B.3. TAC generation uses the data block created through the method above, processed with left-to-right 8 bytes of DTK key in XOR calculation for encryption. MAC or TAC calculation methods please see Figure B.4 for details.

Step 6: 4 bytes on the left of the final value is the MAC or TAC.

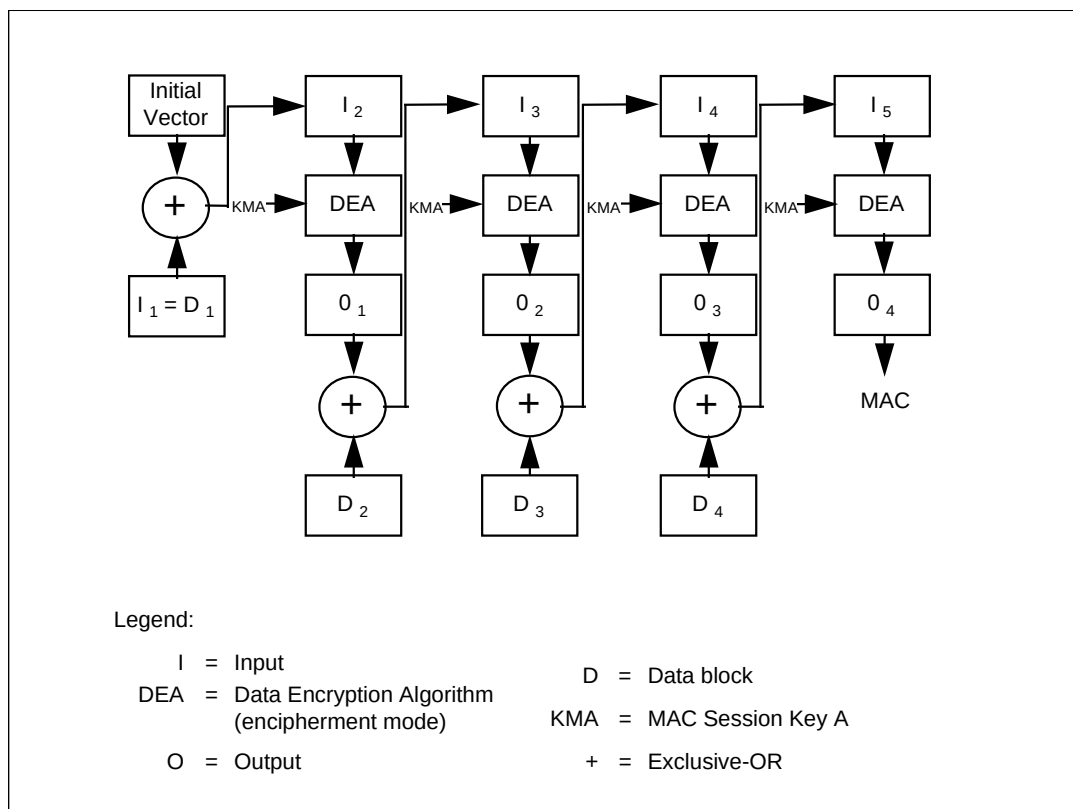


Figure A.1 MAC and TAC single DEA key calculation method